BỘ GIÁO DỤC & ĐÀO TẠO
TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT TP. HỒ CHÍ MINH
KHOA ĐIỆN – ĐIỆN TỬ
BỘ MÔN TỰ ĐỘNG ĐIỀU KHIỂN

ĐỒ ÁN ROBOT

Đề tài

ĐIỀU KHIỂN ROBOT DI ĐỘNG DI CHUYỂN QUA NHIỀU ĐIỂM DỰA TRÊN ROS

GVHD: PGS. TS. Trần Đức Thiện
SVTH: Nguyễn Tấn Tiến
MSSV: 21151171

Tp. Hồ Chí Minh, tháng 06 năm 2025

Lời cảm ơn

Chúng em cũng xin chân thành cảm ơn các thầy cô giáo trong trường Đại Học Sư Phạm Kỹ Thuật TP Hồ Chí Minh nói chung, các thầy cô trong Ngành Kỹ Thuật Điều Khiển Và Tự Động Hóa nói riêng đã dạy cho chúng em kiến thức về các môn đại cương cũng như các môn chuyên ngành, giúp em có được cơ sở lý thuyết vững vàng và tạo điều kiện giúp đỡ em trong suốt quá trình học tập và ứng dụng để thực hiện đồ án môn học.

Đặc biệt, chúng em xin gửi lời cảm ơn chân thành đến thầy PGS. TS. Trần Đức Thiện đã truyền đạt những kiến thức vô cùng hữu ích giúp chúng em hoàn thành đồ án môn học lý thuyết hệ thống điều khiển tự động nâng cao. Thầy là người đã luôn theo dõi, hướng dẫn, chỉ bảo rất tận tình những mặt thiếu sót trong suốt quá trình nhóm thực hiện đề tài.

Trong đề tài chắc chắn sẽ còn nhiều lỗi cần khắc phục, chúng em rất mong nhận được những đóng góp từ phía thầy và các bạn đọc để sửa chữa và mở rộng vốn hiểu biết để hoàn thiện hơn cho những đề tài sau.

TP. Hồ Chí Minh, ngày 19 tháng 06 năm 2025

Sinh viên thực hiện

Nguyễn Tấn Tiến

MỤC LỤC

Lời cảm ơn 1
MỤC LỤC 2
PHỤ LỤC HÌNH ẢNH 4
PHỤ LỤC BẢNG 6
Chương 1: TỔNG QUAN 7
1.1. Đặt vấn đề 7
1.2. Mục tiêu 8
1.3. Giới hạn đề tài 8
1.4. Nội dung thực hiện 8
Chương 2: CƠ SỞ LÝ THUYẾT 9
2.1. Cấu trúc robot di động hai bánh vi sai 9
2.2. Động học robot di động hai bánh vi sai 9
2.2.1. Lý thuyết động học robot di động 9
2.2.2. Động học robot di động hai bánh vi sai 11
2.3. Nền tảng ROS 12
2.3.1. Tổng quan 12
2.3.2. Cấu trúc tổ chức 13
2.4. Tổng quan về điều hướng robot 16
2.5. Lý thuyết cảm biến Lidar 17
2.5.1. Cấu tạo 18
2.5.2. Nguyên lý hoạt động 18
2.6. Lý thuyết cảm biến Encoder 19
2.6.1. Cấu tạo 19
2.6.2. Nguyên lý hoạt động 19
2.7. Lý thuyết cảm biến IMU 20
2.8. Giao thức truyền thông 21
2.8.1. Giao thức UART 21
2.8.2. Giao thức SSH 22
2.9. Lý thuyết bộ điều khiển PID 23
Chương 3: THIẾT KẾ HỆ THỐNG 24
3.1. Cấu hình hệ thống robot di động hai bánh vi sai 24
3.2. Thiết kế mô hình robot trên Solidworks 25
3.2.1. Tổng quan mô hình 25
3.2.2. Thiết kế từng phần 26
3.3. Lựa chọn thiết bị 27
3.3.1. Động cơ 27
3.3.2. Máy tính nhúng 28

3.3.3. Vi điều khiển 28
3.3.4. Cảm biến Lidar 29
3.3.5. Nguồn cấp 30
Chương 4: XÂY DỰNG MÔI TRƯỜNG MÔ PHỎNG 31
4.1 Sử dụng công cụ SolidWorks to URDF Exporter 31
4.2. Mô phỏng robot trên Gazebo 33
4.2.1. Xây dựng không gian hoạt động cho robot 33
4.2.2. Thông số cảm biến lidar 34
4.3. Quan sát robot thông qua Rviz 35
4.4. Thiết kế giao diện điều khiển và thu thập dữ liệu bằng PyQt5 38
Chương 5: THI CÔNG MÔ HÌNH 40
5.1. Sơ đồ mạch điện 40
5.2. Lắp ráp phần cứng 41
5.3. Giao tiếp dữ liệu 41
5.3.1. Giao tiếp giữa Arduino và Raspberry 41
5.3.2. Giao tiếp giữa Raspberry và VMware 45
5.3.3. Kết nối Raspberry với cảm biến Lidar 47
(Chưa nhận được dữ liệu) 47
Chương 6: THỰC NGHIỆM VÀ KẾT LUẬN 48
6.1. Thực nghiệm điều khiển bằng lệnh đơn 48
6.2. Thực nghiệm điều khiển bằng 2D Nav Goal 51
6.3. Thực nghiệm điều khiển chạy nhiều điểm mục tiêu 51
6.4. Kết luận 51

PHỤ LỤC HÌNH ẢNH

Hình 1: Cấu trúc mô hình robot di động vi sai hai bánh..... 9
Hình 2: ROS Filesystem..... 13
Hình 3: Cấu trúc của ROS Computation Graph..... 15
Hình 4: Giao tiếp giữa các node ROS. 16
Hình 5: Sơ đồ cấu trúc điều hướng cho robot di động..... 17
Hình 6: Nguyên lý hoạt động của Lidar. 18
Hình 7: Nguyên lý hoạt động của Encoder..... 20
Hình 8: Cấu trúc tổng quan IMU..... 20
Hình 9: Kết nối trong giao thức UART..... 21
Hình 10: Cấu trúc gói dữ liệu trong giao thức UART..... 21
Hình 11: Cấu trúc bộ điều khiển PID..... 23
Hình 12: Sơ đồ tổng quan hệ thống. 24
Hình 13: Thiết kế mô hình robot hai bánh vi sai trên solidworks. 25
Hình 14: Bãn vẽ tổng quan robot hai bánh vi sai..... 25
Hình 15: Bản vẽ 2D tầng 1 base link của robot. 26
Hình 16: Bản vẽ 2D tầng 2 của robot..... 26
Hình 17: Động cơ JGB37-520. 27
Hình 18: Raspberry Pi 4. 28
Hình 19: Vi điều khiển STM32 F103C8T6. 29
Hình 20: Cảm biến HLS-LFCD2. 29
Hình 21: Pin Sạc 18650. 30
Hình 22: Đế pin..... 30
Hình 23: Đặt trục cho mô hình robot. 31
Hình 24: Biến đổi tọa độ cho hệ thống robot. 32
Hình 25: Mở công cụ URDF..... 32
Hình 26: Cấu trúc liên kết cần thiết lập trong..... 32
Hình 27: Robot trong không gian hoạt động mô phỏng bằng Gazebo..... 33
Hình 28: Mô phỏng robot trong môi trường Gazebo. 34
Hình 29: Hiển thị robot và dữ liệu cảm biến lidar trên Rviz..... 34
Hình 30: Thêm các chủ đề cần quan sát vào Rviz. 35
Hình 31: Cửa sổ Rviz hiển thị bản đồ 2D, tín hiệu cảm biến và dữ liệu trạng thái robot khi chưa chạy Move base. 35
Hình 32: Cửa sổ Rviz hiển thị bản đồ 2D trong lúc chạy Move base. 36
Hình 33: Cửa sổ Rviz hiển thị điểm đơn mục tiêu 2D Nav Goal và quỹ dạo tính toán của robot. 36
Hình 34: Cửa sổ Rviz hiển thị điểm mục tiêu Multi - Goal và quỹ đạo mong muốn. 37
Hình 35: Cửa sổ Rviz hiển thị điểm mục tiêu Multi - Goal đã xác nhận và quỹ đạo đã đi được..... 37

Hình 36: Giao diện người điều khiển và giám sát dữ liệu..... 38
Hình 37: Cửa sổ dữ liệu thu thập từ trạng thái của robot..... 38
Hình 38: Bố trí các cửa sổ trong quá trình điều khiển robot. 39
Hình 39: Sơ đồ mạch điện robot. 40
Hình 40: Khung tầng một base_link. 41
Hình 41: Khung tầng hai lidar_link..... 41
Hình 42: Kết nối vi điều khiển và máy tính nhúng bằng cáp USB. 42
Hình 43: Thông báo kết nối dữ liệu thành công. 42
Hình 44: SSH thành công từ máy tính đến Raspberry..... 45
Hình 45: Kết quả chạy thành công node tính odom. 47
Hình 46: Giao diện điều khiển robot. 48
Hình 47: Dữ liệu điều khiển robot đi tới..... 48
Hình 48: Dữ liệu điều khiển robot đi lùi..... 49
Hình 49: Dữ liệu điều khiển robot quay trái (chiều dương góc Yaw của robot). 50
Hình 50: Dữ liệu điều khiển robot quay phải (chiều âm góc Yaw của robot). 50

PHỤ LỤC BẢNG

Bảng 1: Bảng thông số giao thức UART..... 22
Bảng 2: Thông số động cơ. 27
Bảng 3: Thông số Raspberry Pi 5. 28
Bảng 4: Thông số vi điều khiển STM32. 29
Bảng 5: Thông số của cảm biến HLS-LFCD2. 30
Bảng 6: Thông số pin sạc 18650. 30
Bảng 7: Đặt hệ tọa độ cho từng bộ phận của robot. 31
Bảng 8. Thông số lựa chọn cho cảm biến lidar mô phỏng. 34

Chương 1:
TỔNG QUAN

1.1. Đặt vấn đề

Trong bối cảnh cách mạng công nghiệp 4.0, robot đang trở thành một trong những lĩnh vực được đầu tư nghiên cứu ào ạt và ứng dụng rộng rãi trong nhiều ngành nghề. Trong đó, robot di động hai bánh vi sai đóng vai trò quan trọng như là nền tảng được sử dụng phổ biến trong nghiên cứu và ứng dụng thực tế như giao hàng tự động, dịch vụ logistics, và đồ án phát triển các hệ thống AI. Đặc biệt trong quân sự, robot không chỉ thực hiện nhiệm vụ do thám, giám sát và dò bom mìn mà còn hỗ trợ cứu hộ trong các tình huống nguy hiểm, giảm thiểu rủi ro cho con người. Trong công nghiệp, robot được sử dụng để thực hiện các công việc nguy hiểm như phun thuốc trừ sâu, cắt tỉa, thu hoạch và vận chuyển nặng nhọc, giúp con người tránh khỏi việc tiếp xúc với các hóa chất độc hại.

Để robot có thể hoạt động hiệu quả, chúng cần có khả năng định vị và điều hướng chính xác. Một trong những công nghệ quan trọng giúp robot đạt được điều này là LiDAR (Light Detection and Ranging), hay còn gọi là cảm biến quét bằng laser. LiDAR sử dụng ánh sáng laser để đo khoảng cách và tạo ra bản đồ 3D của môi trường xung quanh, giúp robot nhận diện và tránh né các vật cản. Công nghệ này đặc biệt quan trọng đối với các robot di động 2 bánh vi sai, cho phép chúng di chuyển linh hoạt và an toàn trong các không gian phức tạp.

Việc sử dụng cảm biến LiDAR trong các robot di động 2 bánh vi sai còn giúp thu thập dữ liệu môi trường chính xác, từ đó định vị và lập kế hoạch đường đi hiệu quả. Công nghệ này kết hợp với các phương pháp xử lý dữ liệu như xử lý ảnh, cảm biến laser và các thuật toán dead-reckoning để đảm bảo robot hoạt động chính xác trong mọi điều kiện.

ROS là một nền tảng mã nguồn mở với cộng đồng phát triển rộng lớn. Nó hỗ trợ tích hợp dễ dàng các thư viện, gói phần mềm và công cụ xử lý tín hiệu, lập kế hoạch chuyển động, điều khiển, và cảm biến. Điều này giúp bạn dễ dàng tùy chỉnh và mở rộng hệ thống cho robot di động. ROS tích hợp với Gazebo giúp ta có thể kiểm tra thuật toán điều khiển, hệ thống cảm biến, và lập kế hoạch chuyển động trong môi trường mô phỏng trước khi triển khai thực tế.

Vì ROS là mã nguồn mở, bạn không cần chi trả cho giấy phép phần mềm, đồng thời có thể tận dụng các công cụ và gói phần mềm sẵn có để giảm thời gian và chi phí phát triển.

1.2. Mục tiêu

- Thiết kế và thi công mô hình robot di động vi sai hai bánh.
- Xây dựng môi trường mô phỏng robot.
- Điều khiển robot di động dựa vào ROS.

1.3. Giới hạn đề tài

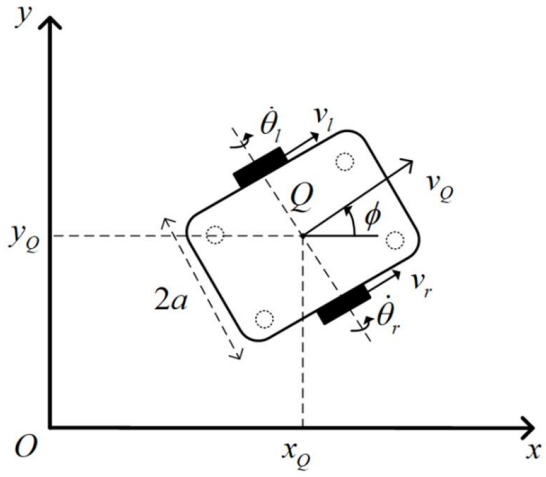
- Đề tài tập trung nghiên cứu mô hình robot di động hai bánh vi sai.
- Các vật cản trong môi trường hoạt động nằm cố định, không thay đổi.
- Giới hạn tốc độ tối đa của robot là 0.15 m/s
- Giới hạn khối lượng tối đa là 5 kg.

1.4. Nội dung thực hiện

- Chương 1: Tổng quan đề tài.
- Chương 2: Cơ sở lý thuyết.
- Chương 3: Thiết kế hệ thống.
- Chương 4: Xây dựng môi trường mô phỏng.
- Chương 5: Thi công mô hình.
- Chương 6: Thực nghiệm và kết luận.

Chương 2:
CƠ SỞ LÝ THUYẾT

2.1. Cấu trúc robot di động hai bánh vi sai



Hình 1: Cấu trúc mô hình robot di động vi sai hai bánh.

Cấu trúc mô hình của robot di động vi sai hai bánh được thể hiện trong Hình 1. Tại đây, (x_l, y_l) đại diện cho vị trí của robot theo hai trục tọa độ, L là khoảng cách giữa hai bánh xe, φ là góc hướng của robot so với trục Ox, v là vận tốc tuyến tính của robot, và v_l, v_r lần lượt là vận tốc tuyến tính của bánh trái và bánh phải. $\dot{\theta}_l$ và $\dot{\theta}_r$ lần lượt là vận tốc góc của bánh trái và bánh phải.

Robot bao gồm hai bánh xe được gắn vào hai bên thân, đóng vai trò truyền động độc lập cho robot. Ngoài ra, robot còn có một bánh đa hướng phía trước để duy trì thăng bằng.

2.2. Động học robot di động hai bánh vi sai

2.2.1. Lý thuyết động học robot di động

Động học robot di động liên quan đến cấu hình robot trong không gian làm việc, mối liên hệ giữa không gian hình học, các ràng buộc trong quỹ đạo của nó. Công thức động học dựa trên cấu trúc hình học của robot. Nghiên cứu về động học của robot là cơ sở để nghiên cứu về phần động lực học, tính ổn định và điều khiển robot. Bài toán động học thuận và động học nghịch tính toán sự biến đổi giữa các biến ở không gian khớp và các biến trong không gian làm việc và ngược lại.

Xét một hệ thống robot di động, không gian khớp chứa các biến tổng quát $q_1, q_2, ..., q_n$, còn không gian làm việc chứa các biến tổng quát $x_1, x_2, ..., x_m$. Các vector này được định nghĩa theo các công thức tương ứng:

$$q = \begin{bmatrix} q_1 \\ q_2 \\ \vdots \\ q_n \end{bmatrix}, \ \mathrm{p} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} \tag{1}$$

Bài toán động học thuận là bài toán tìm **p** khi đã biết trước các biến **q**. Đối với công thức tổng quát $\mathbf{p} \in \Re^m$ và $\mathbf{p} \in \Re^n$ ($\Re^n$ có nghĩa là không gian Euclidean với n chiều). Ta có mối liên hệ bởi hàm phi tuyến theo công thức.

$$p = f(\mathrm{q}), f(q) = \begin{bmatrix} f_1(q) \\ f_2(q) \\ \vdots \\ f_m(q) \end{bmatrix} \tag{2}$$

Để tính toán được các chuyển động của robot ta sẽ xấp xỉ các biến khớp bằng cách tìm vận tốc của các khớp $\dot{\mathbf{q}} = [\dot{\mathrm{q}}_1, \dot{\mathrm{q}}_2, ..., \dot{\mathrm{q}}_n]^T$ .Vì vậy ta cần phải tìm mối liên hệ đạo hàm (vi phân) của **q** và **p** được gọi là động học thuận vi phân và được biểu diễn bởi:

$$dp = Jdq \tag{3}$$

Với:

$$dq = \begin{bmatrix} dq_1 \\ \vdots \\ dq_n \end{bmatrix}, \ dp = \begin{bmatrix} dx_1 \\ \vdots \\ dx_m \end{bmatrix} \tag{4}$$

Và ma trận $m \times n$

$$J = \begin{bmatrix} \frac{\partial x_1}{\partial q_1} & \frac{\partial x_1}{\partial q_2} & \cdots & \frac{\partial x_1}{\partial q_n} \\ \cdots & \cdots & \cdots & \cdots \\ \cdots & \cdots & \cdots & \cdots \\ \frac{\partial x_m}{\partial q_1} & \frac{\partial x_m}{\partial q_2} & \cdots & \frac{\partial x_m}{\partial q_n} \end{bmatrix} = \left[\mathrm{J}_{ij} \right] \tag{5}$$

Ở công thức (5), (i, j) là phần tử, $\mathrm{J}_{ij} = \frac{\partial x_i}{\partial q_j}$ được gọi là ma trận Jacobian của robot, ma trận này biểu thị mối liên hệ giữa vận tốc của khớp với vận tốc của robot trong không gian làm việc. Gọi $\dot{q} = [\dot{\mathrm{q}}_1, ..., \dot{\mathrm{q}}_n]^T$ và $\dot{p} = [\dot{x}_1, \dot{x}_2, ..., \dot{x}_m]^T$ lần lượt là vận tốc của các khớp và không gian làm việc. Chia công thức cho dt ta được phương trình:

$$\frac{dp}{dt} = J\frac{dq}{dt} \text{ hay } \dot{\mathrm{p}} = \mathrm{J}\dot{q} \tag{6}$$

2.2.2. Động học robot di động hai bánh vi sai

Robot di động vi sai thường được biểu diễn bởi 3 đại lượng chính. Các đại lượng được biểu diễn dưới dạng ma trận công thức (7)

$$\mathbf{p} = \begin{bmatrix} x_Q \\ y_Q \\ \phi \end{bmatrix}, \dot{\mathbf{p}} = \begin{bmatrix} \dot{x}_Q \\ \dot{y}_Q \\ \dot{\phi} \end{bmatrix} \tag{7}$$

Giả sử bánh xe lăn không trượt, v_l và v_r lần lượt là vận tốc tuyến tính của bánh trái và vận tốc tuyến tính bánh phải của xe. Dựa vào Hình 1, ta có công thức (8) và (9):

$$v_l = v_Q - a\dot{\phi} \tag{8}$$

$$v_r = v_Q + a\dot{\phi} \tag{9}$$

Cộng và trừ v_l và v_r, ta được công thức (10) và (11):

$$v_Q = \frac{1}{2}\left(v_r + v_l\right) \tag{10}$$

$$2a\dot{\phi} = v_r - v_l \tag{11}$$

Ta tính được sự thay đổi vị trí theo trục tọa độ như sau:

$$\dot{x}_Q = v_Q \cos\phi = \frac{v_r + v_l}{2}\cos\phi \tag{12}$$

$$\dot{y}_Q = v_Q \sin\phi = \frac{v_r + v_l}{2}\sin\phi \tag{13}$$

Từ công thức (11) ta suy ra được:

$$\dot{\phi} = \frac{v_r - v_l}{2a} \tag{14}$$

Ta lại có vị trí góc và vận tốc góc của bánh trái, bánh phải được kí hiệu lần lượt là $\left\{\theta_l \quad \dot{\theta}_l\right\}$, $\left\{\theta_r \quad \dot{\theta}_r\right\}$, và mối liên hệ giữa vận tốc góc và vận tốc dài của bánh trái và phải được tính bằng công thức (15) và (16):

$$v_l = r\dot{\theta}_l \tag{15}$$

$$v_r = r\dot{\theta}_r \tag{16}$$

Dựa vào các biểu thức (12), (13), (15) và (16) ta có mô hình động học của robot hai bánh vi sai được mô tả như sau:

$$\dot{x}_Q = \frac{r}{2}\left(\dot{\theta}_r \cos\phi + \dot{\theta}_l \cos\phi\right) \tag{17}$$

$$\dot{y}_Q = \frac{r}{2}\left(\dot{\theta}_r \sin\phi + \dot{\theta}_l \sin\phi\right) \tag{18}$$

$$\dot{\phi} = \frac{r}{2a}\left(\dot{\theta}_r - \dot{\theta}_l\right) \tag{19}$$

Từ công thức (17), (18) và (19) ta tìm được phương trình động học của phương pháp lái vi sai:

$$\dot{\mathbf{p}} = \begin{bmatrix} \dot{x}_Q \\ \dot{y}_Q \\ \dot{\phi} \end{bmatrix} = \begin{bmatrix} (r/2)\cos\phi \\ (r/2)\sin\phi \\ r/2a \end{bmatrix}\dot{\theta}_r + \begin{bmatrix} (r/2)\cos\phi \\ (r/2)\sin\phi \\ -r/2a \end{bmatrix}\dot{\theta}_l \tag{20}$$

Hay:

$$\dot{\mathbf{p}} = \mathbf{J}\dot{\mathbf{q}} \tag{21}$$

Với:

$$\dot{\mathbf{p}} = \begin{bmatrix} \dot{x}_Q \\ \dot{y}_Q \\ \dot{\phi} \end{bmatrix}, \quad \dot{\mathbf{q}} = \begin{bmatrix} \dot{\theta}_r \\ \dot{\theta}_l \end{bmatrix} \tag{22}$$

Và **J** là ma trận Jacobian.

$$\mathbf{J} = \begin{bmatrix} (r/2)\cos\phi & (r/2)\cos\phi \\ (r/2)\sin\phi & (r/2)\sin\phi \\ r/2a & -r/2a \end{bmatrix} \tag{23}$$

2.3. Nền tảng ROS

2.3.1. Tổng quan

ROS (Robot Operating System) là một khung phần mềm mã nguồn mở, được cấp phép theo giấy phép BSD, nhằm hỗ trợ việc phát triển các ứng dụng robot. Mặc dù không phải là một hệ điều hành thực sự, ROS đóng vai trò như một hệ điều hành meta, cung cấp nhiều tính năng quan trọng cho việc lập trình robot.

Các tính năng của ROS bao gồm giao tiếp giữa các tiến trình thông qua giao diện truyền thông điệp, giúp các chương trình chạy trên ROS có thể liên kết và trao đổi dữ liệu. ROS cũng cung cấp trừu tượng hóa phần cứng, cho phép các nhà phát triển tạo ra các ứng dụng không phụ thuộc vào loại robot cụ thể. Các nút ROS (node) được tổ chức thành các gói, giúp quản lý và xây dựng các ứng dụng dễ dàng hơn.

Khung ROS tích hợp với nhiều thư viện bên thứ ba như OpenCV, PCL, OpenNI, giúp phát triển ứng dụng đa dạng. Nó cũng hỗ trợ điều khiển các thiết

bị cấp thấp và tính toán phân tán, phân phối sức mạnh xử lý để tối ưu hóa hiệu suất. Một mục tiêu quan trọng của ROS là tái sử dụng mã, hỗ trợ lập trình bằng nhiều ngôn ngữ và cung cấp khả năng kiểm thử dễ dàng thông qua rostest.

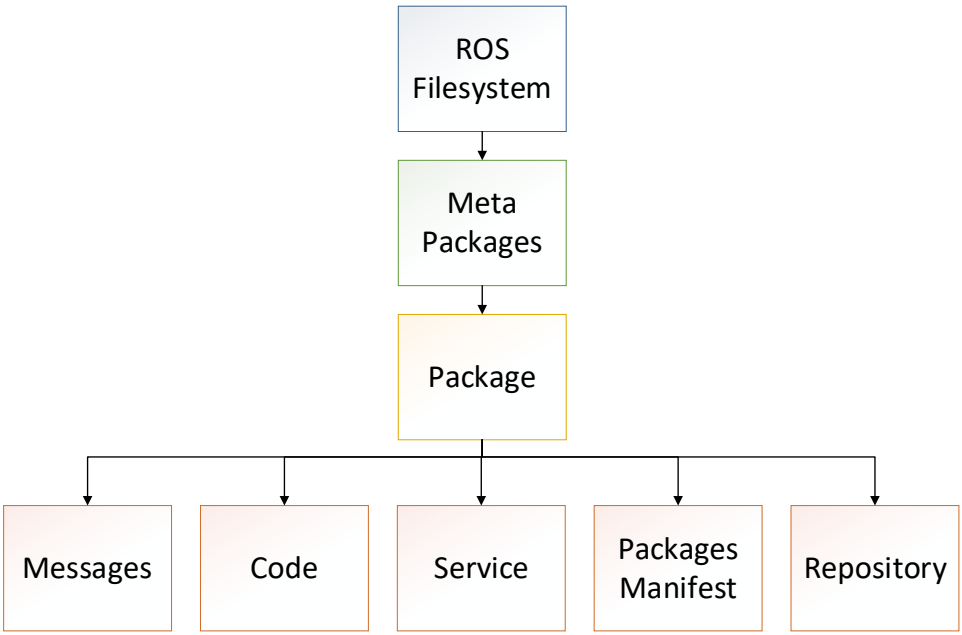
Với khả năng mở rộng và tính linh hoạt, ROS tạo điều kiện phát triển các ứng dụng robot mạnh mẽ. Nó kết hợp các công cụ gỡ lỗi, trực quan hóa dữ liệu, mô phỏng và tích hợp sẵn các khả năng như điều hướng, định vị và kiểm soát robot. Điều này giúp các nhà phát triển dễ dàng tạo ra các ứng dụng robot hiệu quả và toàn diện.

2.3.2. Cấu trúc tổ chức

Có ba cấp tổ chức chính trong ROS

- Hệ thống tệp ROS
- Đồ thị tính toán ROS
- Cộng đồng ROS

*** Hệ thống tệp ROS (ROS Filesystem)** là cấu trúc lưu trữ và quản lý tất cả các tài nguyên phần mềm trong môi trường ROS. Các thành phần cấp độ của ROS filesystem được mô tả ở **Error! Reference source not found.** bên dưới.



Hình 2: ROS Filesystem.

Trong đó:

- Meta packages: Sử dụng cho một nhóm các gói cho một mục đích đặc biệt. Trong các phiên bản cũ còn được gọi cách khác là "Stacks". (Ví dụ như Navigation Stack).

- Packages: ROS Packages là đơn vị cá nhân của khung phần mềm ROS. Một gói ROS có thể chứa mã nguồn, thư viện bên thứ ba, tệp cấu hình, và nhiều hơn nữa. Các gói ROS có thể được tái sử dụng và chia sẻ.

- **Message (.msg):** Mô tả thông điệp được lưu trữ trong thư mục msg của một gói. Các thông điệp ROS là cấu trúc dữ liệu để gửi dữ liệu qua hệ thống truyền thông thông điệp của ROS. Định nghĩa thông điệp được lưu trữ trong một tệp có phần mở rộng .msg.

- Code: Ám chỉ mã nguồn của các chương trình thực thi trong ROS. Đây là các đoạn mã được viết bằng các ngôn ngữ lập trình phổ biến như Python, C++, và Lisp để thực hiện các chức năng cụ thể của robot. Mã nguồn giúp xác định hành vi và khả năng của robot, từ điều khiển động cơ, xử lý dữ liệu cảm biến đến thực hiện các thuật toán định vị và điều hướng. Nó là thành phần cốt lõi để phát triển và triển khai các ứng dụng robot mạnh mẽ trong môi trường ROS.

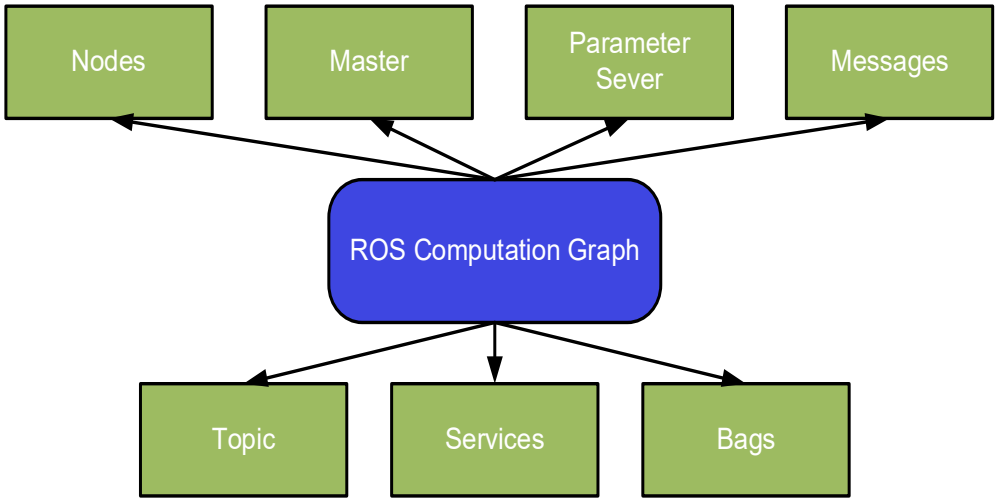
- Service (.srv): Dịch vụ trong ROS được xác định bằng các tệp. srv, nằm trong thư mục srv. Các tệp này mô tả cấu trúc dữ liệu cho các yêu cầu và phản hồi trong dịch vụ.

- Package Manifest: Tệp package.xml chứa tất cả các thông tin chi tiết về gói như tên, mô tả, giấy phép và các phụ thuộc quan trọng khác. Đây là tệp metadata giúp quản lý và phân phối các gói hiệu quả.

- Repository: Kho lưu trữ của các gói, thường được duy trì bằng hệ thống kiểm soát phiên bản như Git, SVN hay Mercurial. Repository giúp quản lý các phiên bản và cộng tác phát triển dễ dàng.

*** Đồ thị tính toán ROS**

Đồ thị tính toán ROS là mạng ngang hàng của các hệ thống ROS xử lý dữ liệu. Các tính năng cơ bản của Đồ thị tính toán ROS bao gồm các nút, ROS Master, tham số máy chủ, các thông điệp và dịch vụ:



Hình 3: Cấu trúc của ROS Computation Graph.

- Nodes (Nút): Nodes là các tiến trình xử lý dữ liệu. Ví dụ, một nút có thể xử lý dữ liệu từ máy quét laser để kiểm tra xem có va chạm không.

- Master: ROS Master cung cấp đăng ký và tra cứu tên cho toàn bộ Đồ thị Tính toán (Computation Graph). Không có Master, các node sẽ không thể tìm thấy nhau, trao đổi thông điệp hoặc gọi các dịch vụ.

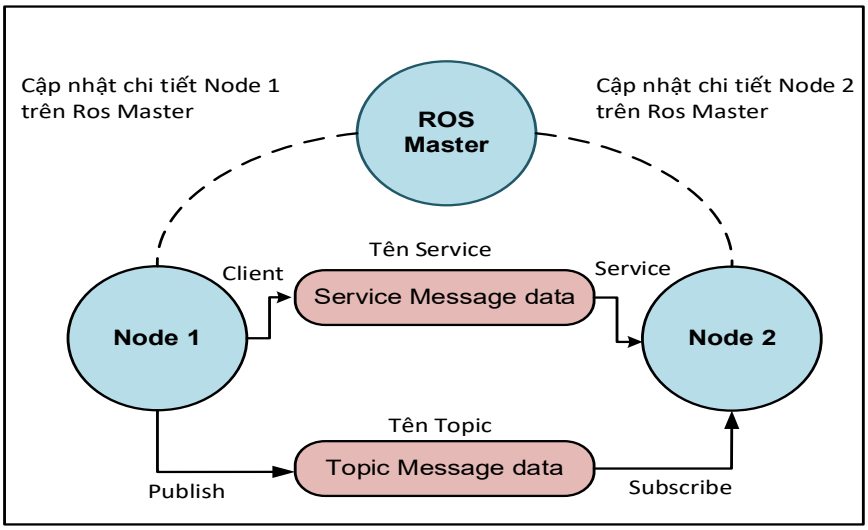
- Parameter Server (Máy chủ tham số): Parameter Server cho phép lưu trữ dữ liệu theo khóa ở vị trí trung tâm. Hiện tại, nó là một phần của Master.

- Messages (Thông điệp): Các node giao tiếp với nhau bằng cách truyền thông điệp. Thông điệp là các cấu trúc dữ liệu dùng để giao tiếp giữa các node.

- Topics (Chủ đề): Là kênh được đặt tên để tạo điều kiện giao tiếp giữa các node. Topics theo mô hình nhắn tin publish-subscribe, cho phép giao tiếp không đồng bộ. Nhiều node có thể publish hoặc subscribe cùng một topic, giúp giao tiếp phân tán và thiết kế hệ thống mô-đun.

- Services (Dịch vụ): Mô hình publish/subscribe là một kiểu giao tiếp rất linh hoạt, nhưng việc truyền tải một chiều từ nhiều nguồn đến nhiều đích không phù hợp cho tương tác yêu cầu/phản hồi, thường cần trong hệ thống phân tán. Yêu cầu/phản hồi được thực hiện qua các dịch vụ, định nghĩa bằng cặp cấu trúc thông điệp: một cho yêu cầu và một cho phản hồi. Một node cung cấp dịch vụ dưới một tên gọi, và một khách hàng sử dụng dịch vụ bằng cách gửi thông điệp yêu cầu và chờ phản hồi. Các thư viện khách hàng ROS thường trình bày tương tác này với lập trình viên như một cuộc gọi thủ tục từ xa (remote procedure call).

- Bags: Bags là định dạng để lưu trữ và phát lại dữ liệu thông điệp ROS.



Hình 4: Giao tiếp giữa các node ROS.

*** Cộng đồng ROS**

Cộng đồng ROS (Robot Operating System) là một mạng lưới toàn cầu gồm các nhà phát triển, nhà nghiên cứu, sinh viên và chuyên gia trong lĩnh vực robot. Cộng đồng này tập trung vào việc chia sẻ kiến thức, tài nguyên, và hỗ trợ lẫn nhau trong việc phát triển và sử dụng ROS. Dưới đây là một số điểm nổi bật của cộng đồng ROS:

- **ROS Answers:** Một diễn đàn hỏi đáp nơi người dùng có thể đặt câu hỏi và nhận câu trả lời từ các chuyên gia khác.

- **Discourse ROS:** Nền tảng thảo luận cho các chủ đề liên quan đến ROS.

- **Mailing Lists:** Các danh sách email nơi các thành viên cộng đồng có thể giao tiếp và cập nhật thông tin.

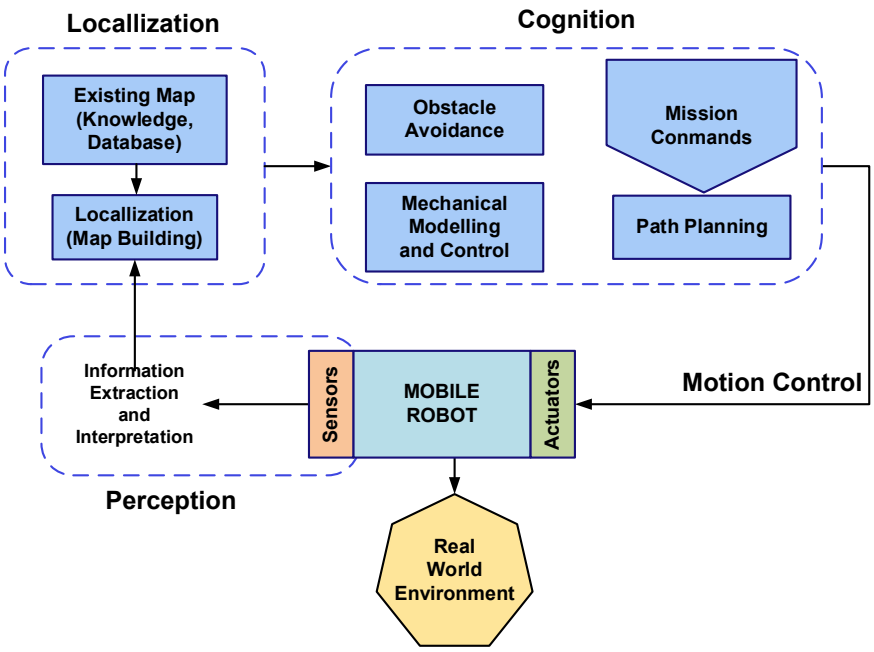
- **Wiki ROS:** Nơi lưu trữ tài liệu và hướng dẫn sử dụng ROS chi tiết.

- **Tutorials và Demos:** Các bài hướng dẫn và minh họa giúp người mới bắt đầu làm quen với ROS.

- **GitHub ROS:** Nơi lưu trữ mã nguồn các dự án ROS, cho phép mọi người đóng góp, sử dụng và cải tiến các gói ROS.

2.4. Tổng quan về điều hướng robot

Điều hướng robot bao gồm việc lập kế hoạch và điều khiển chuyển động từ điểm xuất phát đến điểm đến một cách an toàn và hiệu quả.



Hình 5: Sơ đồ cấu trúc điều hướng cho robot di động.

* **Nhận thức về môi trường (Perception)**: Robot sử dụng các cảm biến như Lidar, camera, IMU, Encoder và siêu âm để thu thập dữ liệu môi trường. Dữ liệu này được sử dụng để xây dựng bản đồ môi trường và nhận diện các vật cản, giúp robot hiểu rõ hơn về không gian xung quanh.

* **Định vị (Localization):** Robot xác định vị trí hiện tại của mình trên bản đồ thông qua các thuật toán như Dead-reckoning, bộ lọc hạt, bộ lọc Kalman và sử dụng cột mốc để cải thiện độ chính xác. Ngoài ra, robot còn có thể thực hiện định vị và xây dựng bản đồ đồng thời bằng giải thuật SLAM (Simultaneous Localization and Mapping).

* **Nhận thức của robot (Cognition):** Khi biết được vị trí hiện tại của mình và các vật cản, robot lập lộ trình tối ưu từ điểm hiện tại đến điểm đích. Các tiêu chí lập kế hoạch bao gồm tối ưu hóa chi phí, năng lượng, thời gian hoặc khoảng cách, đồng thời đảm bảo an toàn cho robot và các vật xung quanh. Các thuật toán như A*, Dijkstra, và RRT được sử dụng cho kế hoạch toàn cục, trong khi DWA (Dynamic Window Approach) và MPC (Model Predictive Control) được dùng cho kế hoạch cục bộ để đảm bảo an toàn khi gặp vật cản động. Robot sau đó tính toán và đưa ra lệnh điều khiển đến các động cơ (Actuators).

2.5. Lý thuyết cảm biến Lidar

Cảm biến Lidar (Light Detection and Ranging) là một công nghệ đo khoảng cách sử dụng ánh sáng laser. Công nghệ này hoạt động bằng cách phát

ra chùm tia laser và đo thời gian chùm tia phản xạ trở lại sau khi va chạm vào các đối tượng xung quanh. Từ đó, Lidar có thể tạo ra một bản đồ 3D chi tiết của môi trường, bao gồm cả khoảng cách và hình dạng của các vật thể.

2.5.1. Cấu tạo

Cảm biến Lidar gồm các thành phần chính:

- Nguồn phát tia laser: Tạo ra các xung ánh sáng laser để quét môi trường.

- Máy thu: Thu nhận các xung laser phản xạ lại từ các bề mặt vật thể.

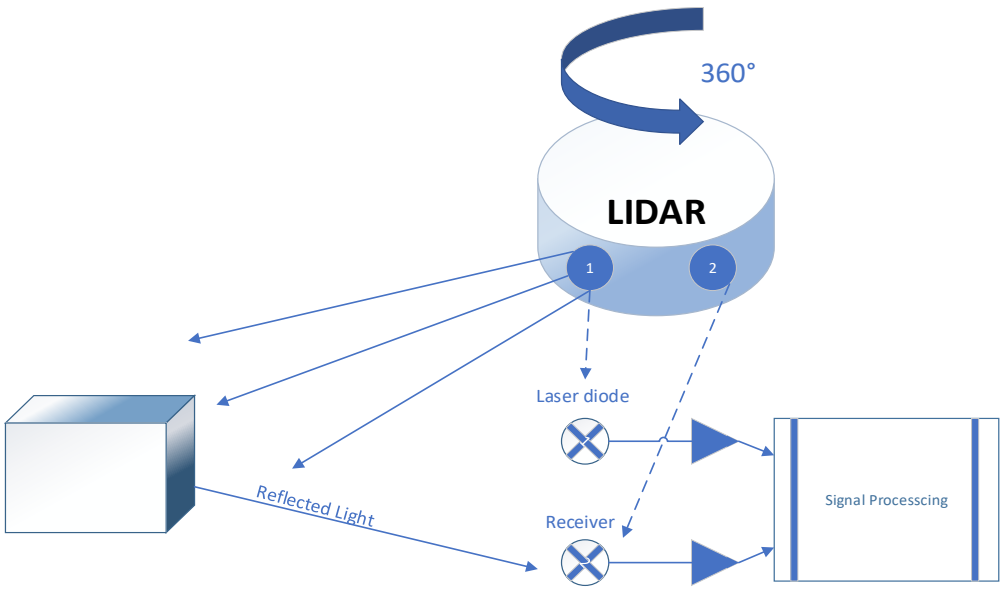
- Hệ thống quét: Điều khiển hướng của tia laser để quét toàn bộ khu vực.

- Bộ xử lý tín hiệu: Tính toán khoảng cách và tạo ra bản đồ 3D.

- Hệ thống định vị và điều khiển (Positioning and Control System)

2.5.2. Nguyên lý hoạt động

Cảm biến Lidar hoạt động bằng cách phát ra các xung ánh sáng laser ngắn, thường ở tần số cao. Khi các xung này va chạm với bề mặt của vật thể trong môi trường, chúng sẽ phản xạ lại. Máy thu trong cảm biến Lidar sẽ nhận các tia laser phản xạ này và đo thời gian từ khi phát ra đến khi nhận lại tín hiệu. Dựa trên thời gian phản hồi, hệ thống tính toán khoảng cách giữa cảm biến và vật thể. Dữ liệu khoảng cách từ các xung laser phản xạ được tổng hợp lại để tạo ra bản đồ 3D chi tiết về môi trường xung quanh. Nhờ đó, Lidar cung cấp thông tin chi tiết về khoảng cách và hình dạng của các vật thể, giúp robot hoặc hệ thống tự hành có thể xác định và điều hướng một cách chính xác trong không gian.



Hình 6: Nguyên lý hoạt động của Lidar.

2.6. Lý thuyết cảm biến Encoder

Cảm biến Encoder là thiết bị dùng để biến đổi vị trí hoặc chuyển động cơ học thành tín hiệu điện tử. Encoder có nhiệm vụ phát hiện vị trí, tốc độ và hướng của một vật thể di chuyển. Chúng thường được sử dụng trong các ứng dụng như robot, máy công cụ, hệ thống tự động hóa, và thiết bị y tế.

Các loại cảm biến Encoder phổ biến bao gồm:

- Encoder quay (Rotary Encoder).

- Encoder tuyến tính (Linear Encoder).

2.6.1. Cấu tạo

Cảm biến Encoder thường có cấu tạo gồm các thành phần chính sau:

- Đĩa mã hóa (Encoder Disk): có các lỗ, vùng phân biệt bằng quang học.

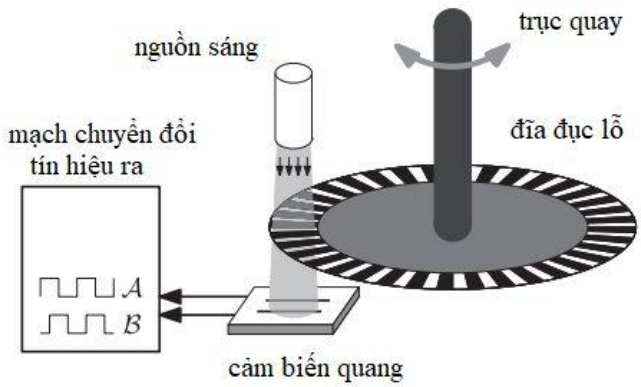
- Đầu đọc (Sensor Head): Bao gồm cảm biến quang học hoặc từ tính, nó sẽ phát hiện các thay đổi trên đĩa mã hóa. Đầu đọc quang học sử dụng ánh sáng để phát hiện các lỗ hoặc vạch trên đĩa, trong khi đầu đọc từ tính sử dụng từ trường.

- Bộ phát tín hiệu (Signal Processing Circuit): Bộ phận này nhận tín hiệu từ đầu đọc và chuyển đổi chúng thành tín hiệu điện tử mà các thiết bị điều khiển khác có thể sử dụng.

- Trục quay (Shaft): Trục này kết nối đĩa mã hóa với vật thể cần đo. Khi vật thể di chuyển, trục quay và làm thay đổi vị trí của đĩa mã hóa.

2.6.2. Nguyên lý hoạt động

Khi vật thể di chuyển, trục quay của Encoder cũng quay theo. Đầu đọc quang học hoặc từ tính sẽ phát hiện các lỗ hoặc vạch trên đĩa mã hóa khi nó quay. Đầu đọc chuyển các thay đổi này thành tín hiệu điện tử. Với Encoder quang học, ánh sáng từ đầu đọc sẽ bị gián đoạn bởi các lỗ hoặc vạch, tạo ra xung tín hiệu. Với Encoder từ tính, từ trường sẽ thay đổi theo các vạch từ tính trên đĩa. Tín hiệu từ đầu đọc được gửi đến bộ phát tín hiệu, nơi chúng được xử lý và chuyển đổi thành dạng tín hiệu mà hệ thống điều khiển có thể sử dụng.

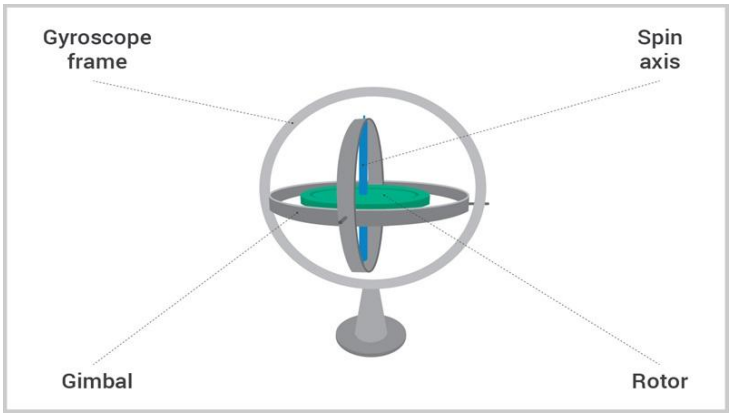


Hình 7: Nguyên lý hoạt động của Encoder.

2.7. Lý thuyết cảm biến IMU

Cảm biến IMU (Inertial Measurement Unit) là thiết bị đo lường quán tính, sử dụng để đo các lực gia tốc và tốc độ góc của một đối tượng trong không gian. IMU thường được sử dụng trong các ứng dụng như định vị, dẫn đường, ổn định của phương tiện, các hệ thống điều khiển tự động.

Cấu tạo cảm biến IMU



Hình 8: Cấu trúc tổng quan IMU.

IMU thường bao gồm hai loại cảm biến chính:

- Gia tốc kế (Accelerameter): Đo gia tốc tuyến tính do các trục x, y và z.
- Con quay hồi chuyển (Gyroscope): Đo tốc độ góc theo các trục x, z và z.

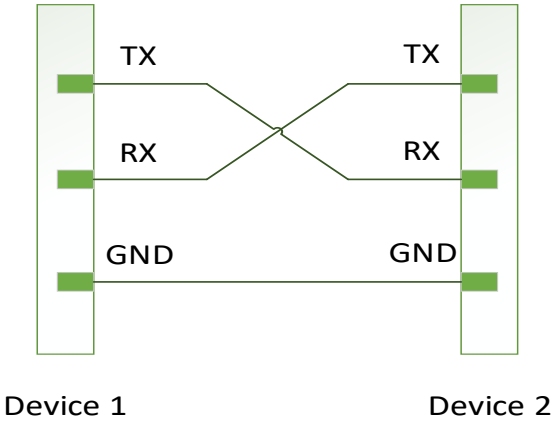
Khi được kết hợp thêm từ kế (Magnetometer), cảm biến IMU có thể được gọi là IMMU ((Inertial Measurement and Magnetometer Unit).

- Từ kế (Magnetometer): Đo từ trường xung quanh để cung cấp thông tin về hướng tương đối theo từ trường trái đất.

2.8. Giao thức truyền thông

2.8.1. Giao thức UART

Giao thức UART (Universal Asynchronous Receiver/Transmitter) là một giao thức truyền thông nối tiếp không đồng bộ.

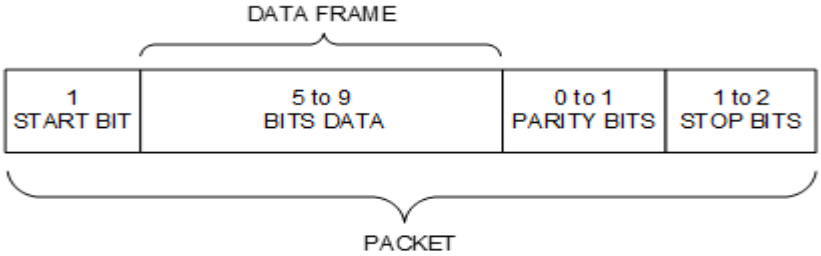


Hình 9: Kết nối trong giao thức UART.

*** Nguyên lý hoạt động:**

- Gửi và nhận dữ liệu: UART chuyển đổi dữ liệu từ dạng song song sang dạng nối tiếp để gửi qua các đường truyền và ngược lại từ nối tiếp sang song song để nhận.

- Khung dữ liệu (Data Frame): Dữ liệu được gửi đi dưới dạng các khung, bao gồm bit bắt đầu (start bit), các bit dữ liệu, bit chẵn lẻ (parity bit) (tuỳ chọn), và bit dừng (stop bit).



Hình 10: Cấu trúc gói dữ liệu trong giao thức UART.

- Tốc độ truyền (Baud Rate): Tốc độ truyền dữ liệu được quy định bằng baud rate, là số bit truyền đi trong một giây. Cả hai UART phải hoạt động ở tốc độ baud tương tự nhau. Tốc độ baud giữa UART truyền và UART nhận chỉ có thể khác nhau khoảng 10% trước khi thời gian của các bit bị sai lệch quá nhiều. Cả hai UART cũng phải được cấu hình để truyền và nhận cùng một cấu trúc gói dữ liệu.

Bảng 1: Bảng thông số giao thức UART.

Số dây sử dụng	2
Tốc độ	9600, 19200, 38400, 57600, 115200
Phương thức truyền thông	Không đồng bộ
Phương pháp kết nối	Nối tiếp
Số lượng tối đa của Master	1
Số lượng tối đa của Slave	1

***Ứng dụng:**

- Giao tiếp giữa các vi điều khiển: UART thường được sử dụng để trao đổi dữ liệu giữa các vi điều khiển, giữa vi điều khiển và các thiết bị ngoại vi.

- Kết nối với máy tính: UART cũng được sử dụng để giao tiếp giữa các thiết bị nhúng và máy tính thông qua cổng nối tiếp (serial port).

- UART là một giao thức phổ biến và dễ triển khai, nhờ vào khả năng không đồng bộ, không yêu cầu xung nhịp đồng bộ (synchronization clock) giữa thiết bị gửi và nhận. Điều này giúp tiết kiệm tài nguyên và giảm phức tạp trong thiết kế hệ thống.

2.8.2. Giao thức SSH

Giao thức SSH (Secure Shell) là một giao thức mạng được thiết kế để cung cấp một phương thức bảo mật cho việc truyền tải dữ liệu qua một mạng không an toàn, như Internet. SSH chủ yếu được sử dụng để điều khiển từ xa các hệ thống, quản lý máy chủ và thực hiện các tác vụ bảo mật, bao gồm đăng nhập từ xa, sao chép tập tin, và chạy các lệnh trên các hệ thống từ xa.

***Các đặc điểm chính của giao thức SSH:**

- Bảo mật cao: SSH mã hóa toàn bộ dữ liệu giữa máy khách và máy chủ, giúp bảo vệ thông tin khỏi các cuộc tấn công như nghe lén (eavesdropping) và sửa đổi dữ liệu.

- Xác thực người dùng: SSH sử dụng các phương thức xác thực mạnh, thường là bằng cách sử dụng khóa công khai (public key) hoặc mật khẩu.

- Khả năng mã hóa: Tất cả dữ liệu, bao gồm các lệnh và dữ liệu đầu ra, đều được mã hóa để đảm bảo rằng thông tin không bị tiết lộ trên đường truyền.

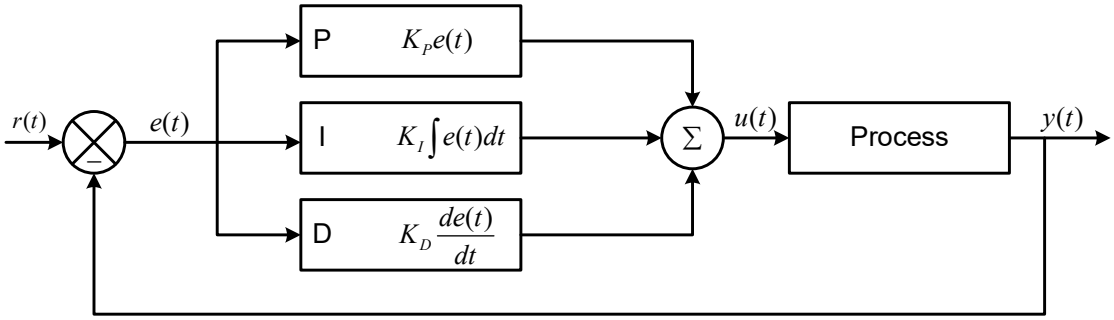
***Các thành phần chính trong SSH:**

- SSH Client: Là ứng dụng hoặc phần mềm cài đặt trên máy tính của người dùng để kết nối đến máy chủ qua giao thức SSH. Một số ví dụ phổ biến là OpenSSH (trên Linux, macOS) hoặc PuTTY (trên Windows).

- SSH Server: Là phần mềm chạy trên máy chủ mà người dùng kết nối tới, ví dụ như sshd (SSH Daemon) trong hệ điều hành Linux hoặc Unix.

2.9. Lý thuyết bộ điều khiển PID

Bộ điều khiển vi tích phân tỉ lệ (Proportional Integral Derivative – PID) bao gồm 3 khâu: khâu tỉ lệ (Proportional), khâu tích phân (Integrated) và khâu vi phân (Derivative). PID là một bộ điều khiển đơn giản, được sử dụng nhiều trong thực tế để điều khiển các hệ thống. Bộ điều khiển PID có khả năng điều chỉnh sai số thấp nhất có thể, tăng tốc độ đáp ứng, giảm độ vọt lố, hạn chế sự dao động.. Có 3 thông số để điều chỉnh tín hiệu điều khiển: K_P, K_I, K_D.



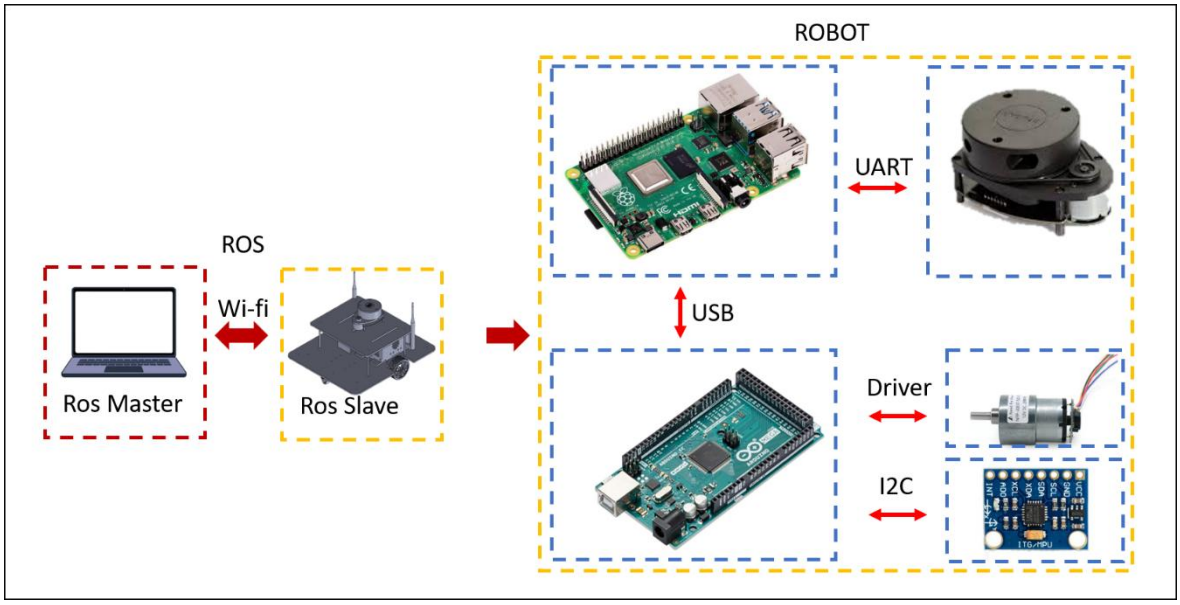
Hình 11: Cấu trúc bộ điều khiển PID.

Trong đó, *r(t)* là giá trị đặt, *y(t)* là giá trị ngõ ra, giá trị ngõ ra được hồi tiếp về để so sánh với giá trị đặt *r(t)* . Sự sai lệch giữa tín hiệu đặt và ngõ ra gọi là sai số *e(t)*. Sai số được đưa vào bộ điều khiển PID sau đó xuất ra tín hiệu điều khiển *u(t)* điều khiển hệ thống Process.

Các thông số K_P, K_I, K_D là các thông số của bộ điều khiển PID cần phải lựa chọn. Nếu thông số này được lựa chọn phù hợp thì hệ thống sẽ hoạt động tốt và chính xác, có nghĩa là độ vọt lố thấp, thời gian xác lập nhanh và sai số xác lập thấp.

Chương 3:
THIẾT KẾ HỆ THỐNG

3.1. Cấu hình hệ thống robot di động hai bánh vi sai



Hình 12: Sơ đồ tổng quan hệ thống.

Người dùng sử dụng máy tính để điều khiển, quan sát kết quả và ra lệnh cho robot di chuyển đến vị trí thông qua máy tính nhúng Raspberry pi 4. Cả hai thiết bị được cài đặt hệ điều hành ROS trên Ubuntu 20.04 và kết nối chung trong một hệ thống wifi, giao tiếp qua chuẩn SSH, trong đó Raspberry được cấu hình IP tĩnh. Máy tính nhúng nhận tín hiệu về từ cảm biến Lidar để xây dựng bản đồ. Vi điều khiển nhận dữ liệu từ máy tính nhúng và điều khiển cơ cấu chấp hành gồm 2 động cơ DC.

Mô hình được phân cấp như sau:

*** Máy tính điều khiển (PC cài đặt Ubuntu 20.04, ROS)**

- Người dùng gửi lệnh iều khiển từ xa. Quan sát bản đồ, vị trí của robot.
- Kết nối với Raspberry Pi 4 qua SSH.

*** Máy tính nhúng (Raspberry Pi 4 cài đặt Ubuntu 20.04, ROS)**

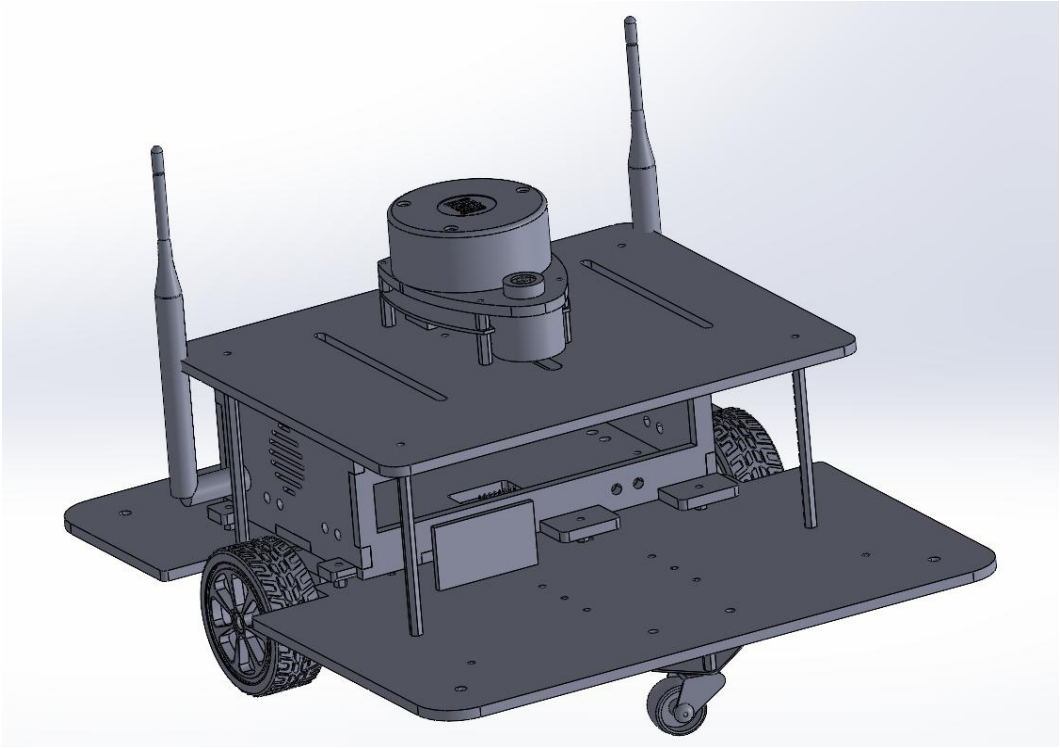
- Nhận lệnh từ PC thông qua mạng WiFi (SSH, ROS).
- Nhận dữ liệu từ LiDAR để định vị (AMCL).
- Gửi tín hiệu điều khiển đến vi điều khiển.

*** Vi điều khiển (Arduino Mega Atmega2560)**

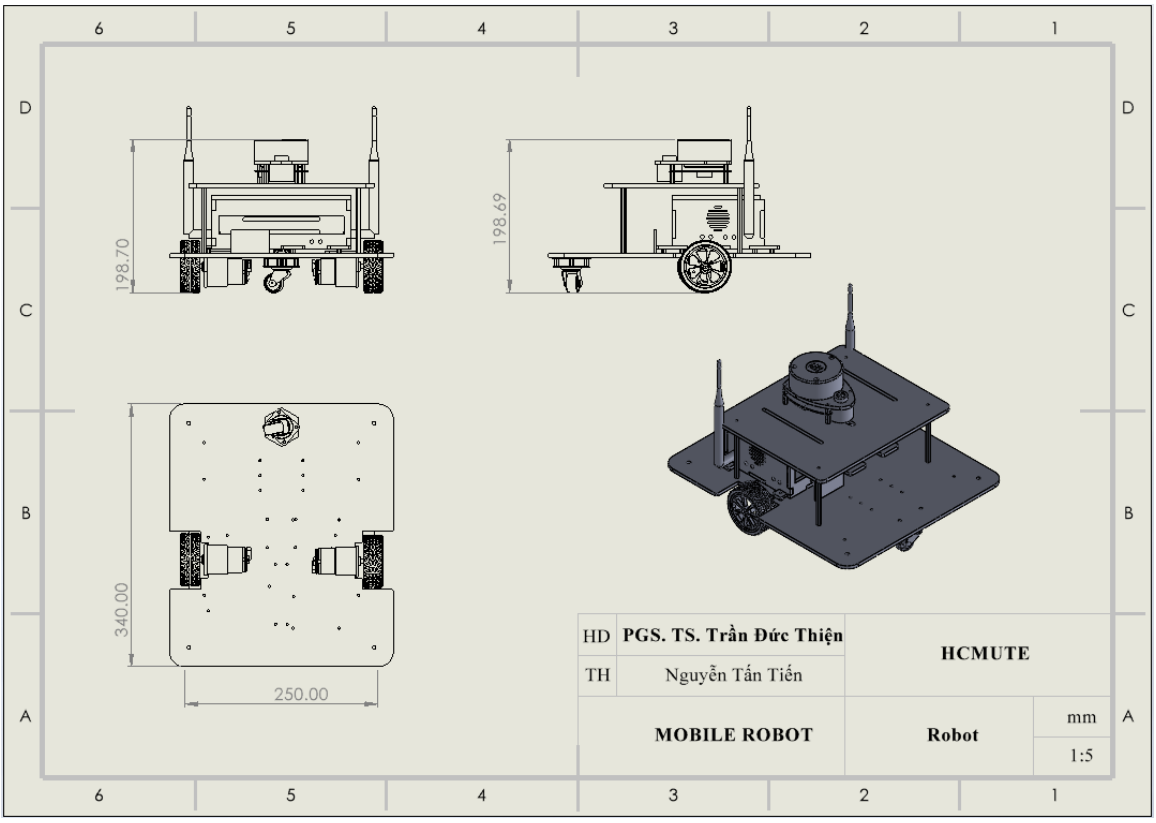
- Nhận lệnh từ Raspberry Pi 4.
- Điều khiển động cơ DC để di chuyển robot.

3.2. Thiết kế mô hình robot trên Solidworks

3.2.1. Tổng quan mô hình



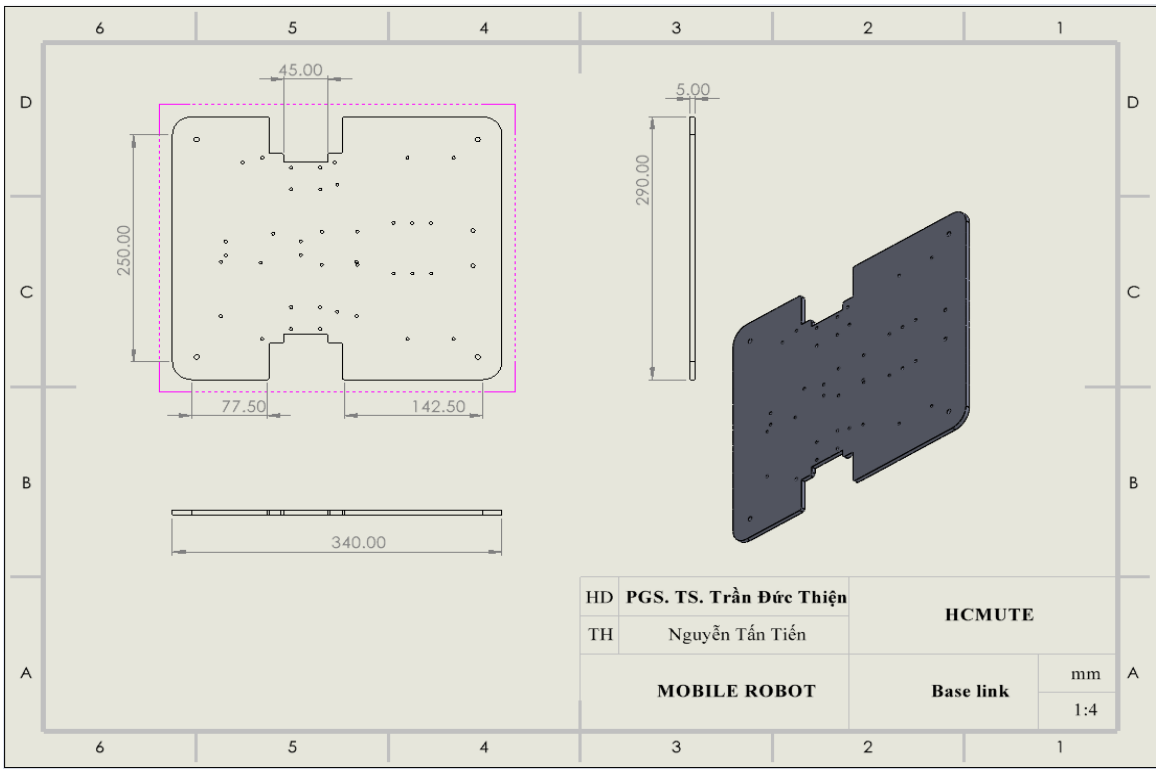
Hình 13: Thiết kế mô hình robot hai bánh vi sai trên solidworks.



Hình 14: Bãn vẽ tổng quan robot hai bánh vi sai.

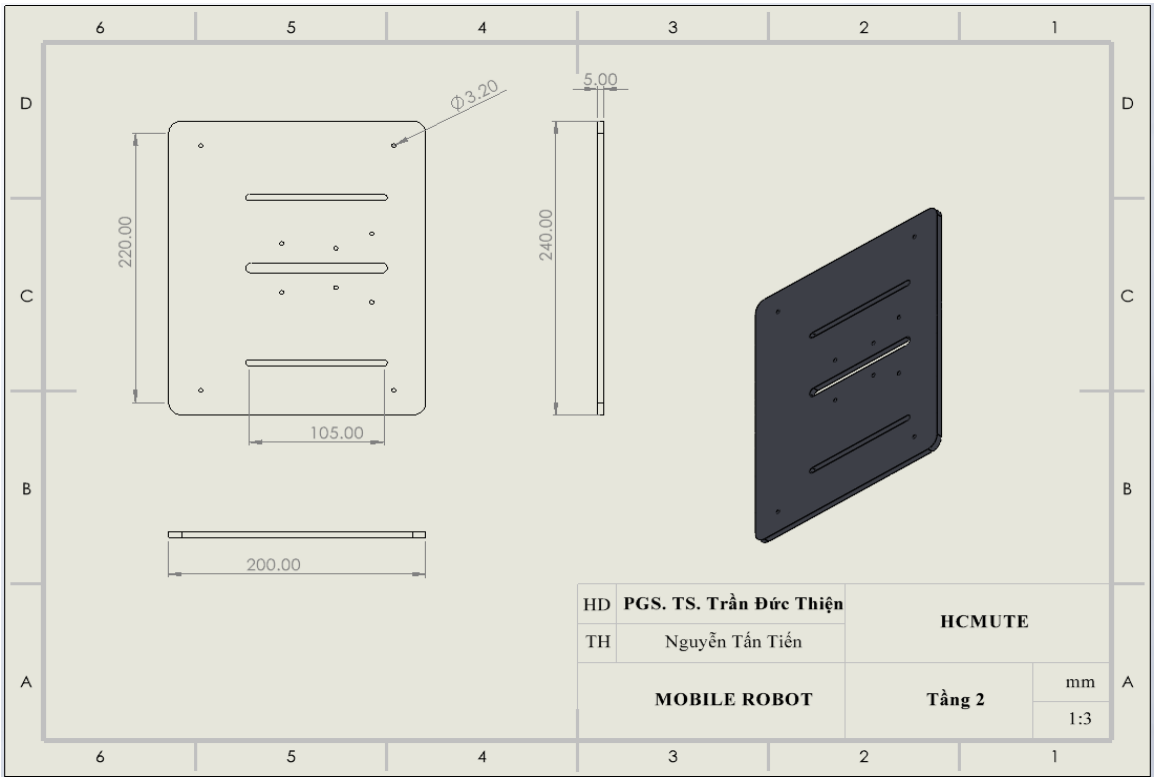
3.2.2. Thiết kế từng phần

- Thiết kế Base link:



Hình 15: Bản vẽ 2D tầng 1 base link của robot.

- Thiết kế tầng hai:



Hình 16: Bản vẽ 2D tầng 2 của robot.

3.3. Lựa chọn thiết bị

3.3.1. Động cơ

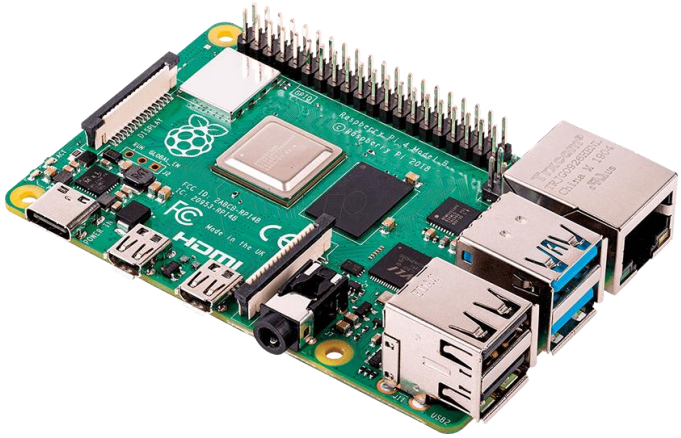
Hình 17: Động cơ JGB37-520.

Bảng 2: Thông số động cơ.

Thông số	**Giá trị**
Tốc độ quay	130 rpm
Điện áp định mức	12 V dc
Điện áp hoạt động	6-12 V dc
Loại	Có chổi than
Đường kính trục	4mm
Chiều dài động cơ	55mm
Đường kính động cơ	25mm
Tỉ số truyền	1:45
Xung/vòng/kênh	22 xung
Chiều dài trục	8mm

3.3.2. Máy tính nhúng

Vì sử dụng hệ thống ROS để điều khiển robot di động, nên nhóm cần 1 máy tính nhúng chạy hệ điều hành để giao tiếp với ROS, nhóm chọn Raspberry Pi 4.



Hình 18: Raspberry Pi 4.

Bảng 3: Thông số Raspberry Pi 5.

Thông số	**Giá trị**
Điện áp định mức	5VDC
Dòng điện định mức	5A
USB	USB-C (Power Delivery), 2 x USB 3.0, 2 x USB 2.0
CPU	Broadcom BCM2712, quad-core Arm Cortex A76 @ 2.4GHz
GPU	VideoCore VII
Memory	LPDDR4X-4267 SDRAM 4 GB 64-bit
Others	Gigabit Ethernet, Wi-Fi 802.11ac, Bluetooth 5.0, microSD card slot, 2 x USB 3.0, 2 x USB 2.0, PCIe 2.0 x1 interface

3.3.3. Vi điều khiển

Hệ thống robot cần đảm nhiệm các nhiệm vụ như đo lường, thu thập dữ liệu từ cảm biến, áp dụng bộ lọc để xử lý dữ liệu thô, truyền dữ liệu và điều khiển động cơ. Để đáp ứng các chức năng này, nhóm đã chọn vi điều khiển Atmega2560.

Hình 19: Vi điều khiển STM32 F103C8T6.

Bảng 4: Thông số vi điều khiển STM32.

Thông số kỹ thuật	**Giá trị**
Điện áp hoạt động	5 VDC
Bộ nhớ Flash	256 kB
SRAM	8 kB
EEPROM	4 kB
Xung Clock	16 MHz

3.3.4. Cảm biến Lidar

Cảm biến Lidar được sử dụng trong việc xác định vị trí của robot, phát hiện vật cản cũng như xây dựng bản đồ. Trong hệ thống mô hình robot, nhóm lựa chọn sử dụng cảm biến HLS-LFCD2 Hitachi.

Hình 20: Cảm biến HLS-LFCD2.

Bảng 5: Thông số của cảm biến HLS-LFCD2.

Thông số	**Giá trị**
Nguồn cấp cho động cơ	5 VDC
Nguồn cấp cho Scanner	5 VDC
Khoảng cách quét	0.12 – 3,5 m
Phạm vi góc quét	0 – 360 độ
Tần số lấy mẫu	1,8 kHz
Tần số quét	1 – 10 Hz

3.3.5. Nguồn cấp

Để cấp nguồn cho động cơ 12V và máy tính nhúng, vi điều khiển, nhóm đã chọn sử dụng pin 18650 với phương pháp nối tiếp.

Hình 21: Pin Sạc 18650.

Hình 22: Đế pin.

Bảng 6: Thông số pin sạc 18650.

Thông số	**Giá trị**
Dung lượng	2600 mAh
Điện áp	3.7 V
Dòng xả liên tục	20 A

Đi kèm với pin cell là đế pin, nhóm lựa chọn đế pin mô tả ở Hình 22, được cấu tạo nối tiếp để đáp ứng đủ dòng điện cho các thiết bị.

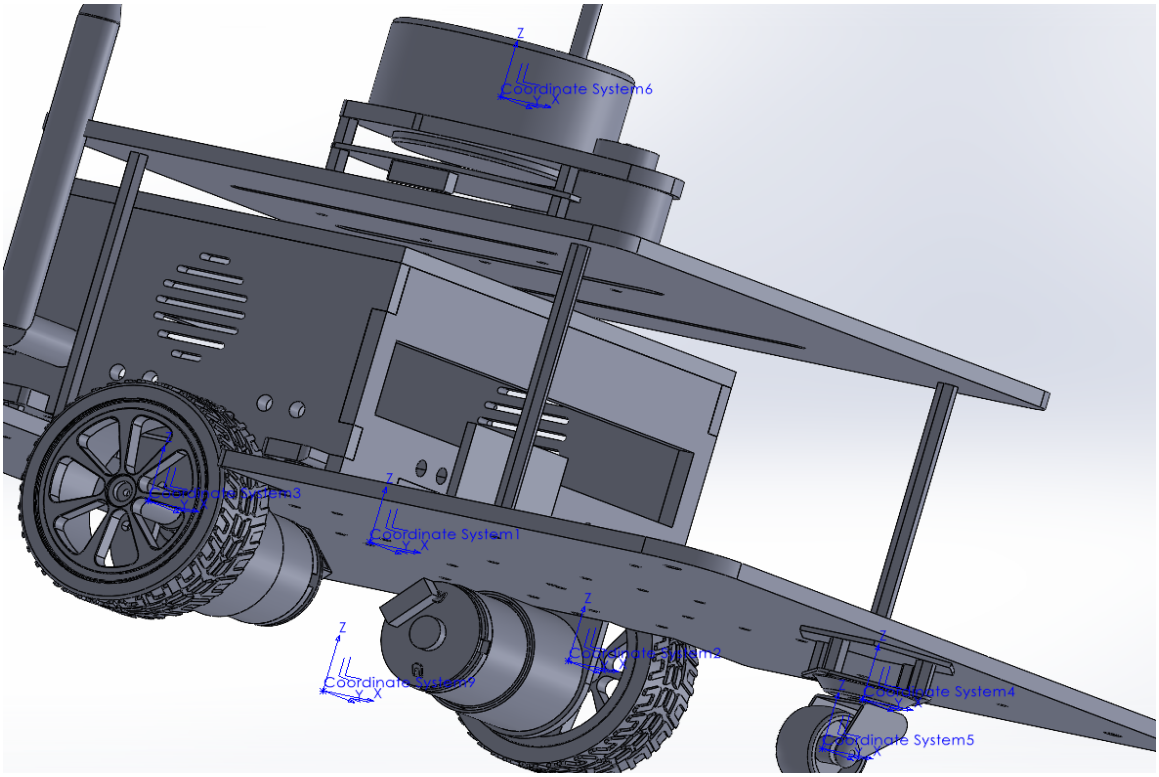
Chương 4:
XÂY DỰNG MÔI TRƯỜNG MÔ PHỎNG

4.1 Sử dụng công cụ SolidWorks to URDF Exporter

SolidWorks to URDF Exporter là một tiện ích mở rộng của phần mềm SolidWorks, giúp chuyển đổi các bộ phận lắp ráp từ SolidWorks sang định dạng URDF. Công cụ này tạo ra một gói tương thích với ROS, bao gồm một thư mục chứa các tệp lưới (meshes), kết cấu (textures) và tệp mô tả robot (URDF).

*** Sử dụng công cụ Solidworks to URDF Export .**

- Bước 1: Đặt các hệ trục tọa độ cho các thành phần của robot.



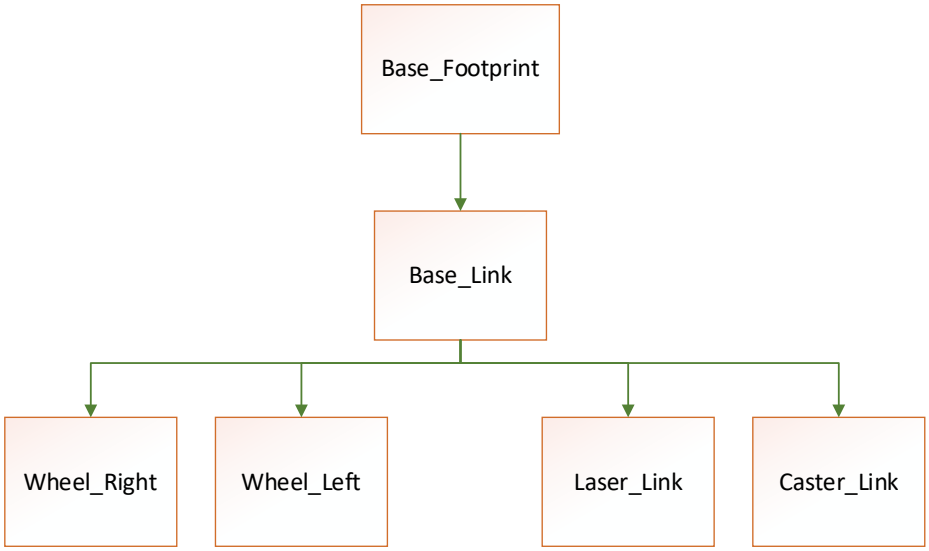
Hình 23: Đặt trục cho mô hình robot.

Trong đó, ta đặt tương ứng các hệ trục Coordinate System như bảng sau:

Bảng 7: Đặt hệ tọa độ cho từng bộ phận của robot.

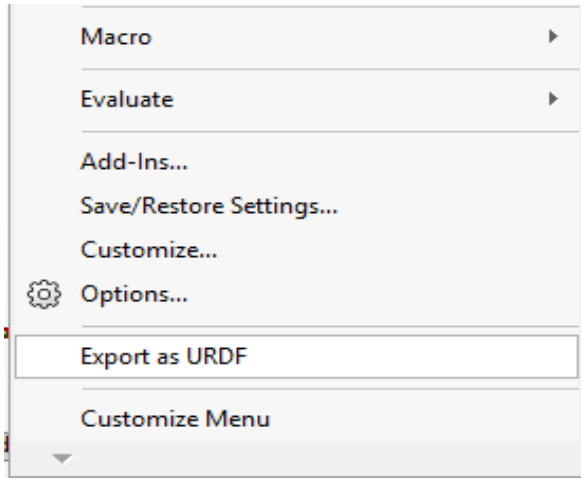
Coordinate System 1	Gốc đặt base_link.
Coordinate System 2	Khớp xoay bánh trái.
Coordinate System 3	Khớp xoay bánh phải.
Coordinate System 4	Khớp xoay bánh phụ trên.
Coordinate System 5	Khớp xoay bánh phụ dưới.
Coordinate System 6	Gốc đặt cảm biến lidar.
Coordinate System 9	Gốc đặt base_footprint

- Bước 2: Thiết lập các quan hệ liên kết cho các cơ cấu.

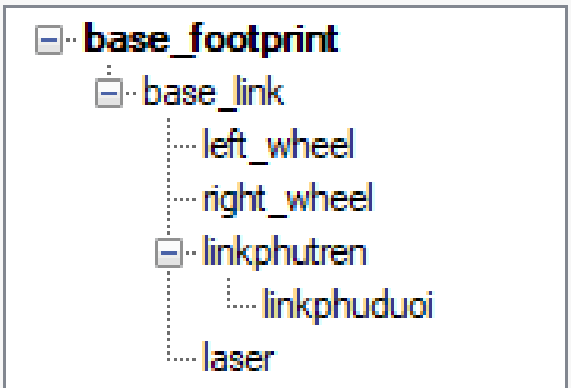


Hình 24: Biến đổi tọa độ cho hệ thống robot.

Tiếp tục, vào "Tools" và chọn "Export as URDF". Tiếp theo, ta thiết lập các quan hệ liên kết như đã xác định trong Hình 24. Sau khi hoàn thành, chọn "Preview and Export". Tiếp tục chọn "Next" ở góc phải cuối màn hình.



Hình 25: Mở công cụ URDF.



Hình 26: Cấu trúc liên kết cần thiết lập trong.

Cuối cùng, chọn “Export URDF and Meshes...”

- Bước 3: Kiểm tra file .urdf vừa tạo.

+ Truy cập Link kiểm chứng, sau đó kéo file URDF vừa xuất thả vào:

https://gkjohnson.github.io/urdf-loaders/javascript/example/bundle/index.html

+ Chọn show controls.

+ Chọn Ignore Joint Limits.

+ Tiến hành kiểm tra đủ số khâu (link): Mỗi link 1 màu.

+ Tiến hành kiểm tra đủ số khớp: Mỗi khớp 1 thanh điều khiển.

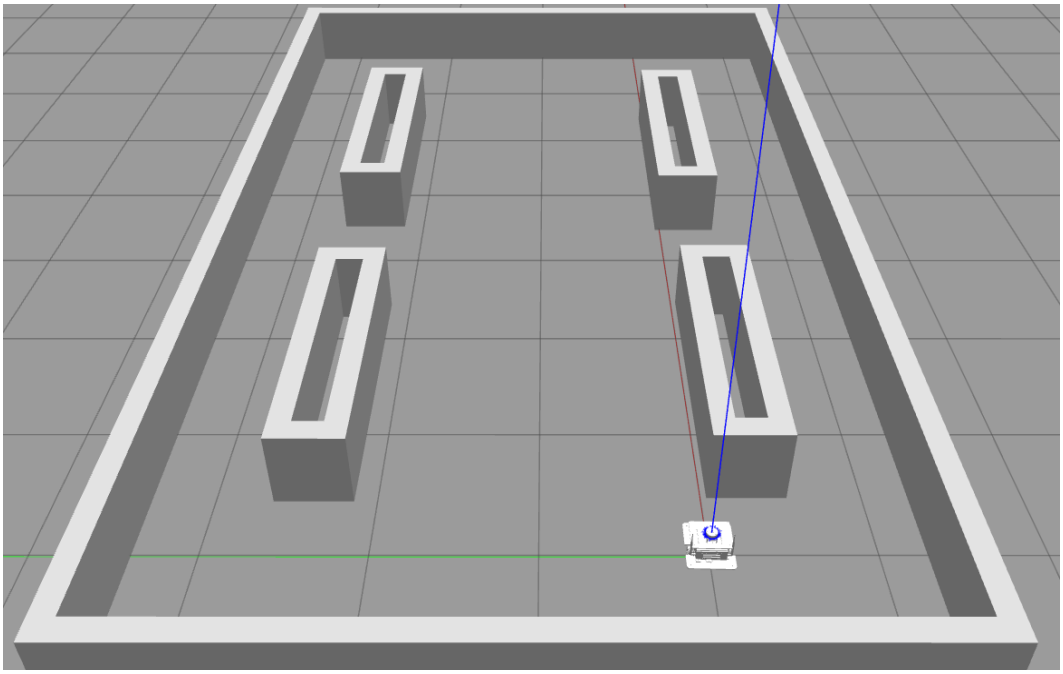
+ Tiến hành kiểm tra chuyển động: Kéo thanh điều khiển, quan sát.

4.2. Mô phỏng robot trên Gazebo

4.2.1. Xây dựng không gian hoạt động cho robot

Môi trường mô phỏng trong Gazebo được thiết kế dựa trên không gian hoạt động thực tế để có thể ước lượng, tính toán và mô phỏng dữ liệu một cách chính xác và hiệu quả hơn. Điều này giúp đảm bảo các thí nghiệm và kiểm thử với robot trong môi trường ảo phản ánh chính xác các điều kiện thực tế, từ đó cải thiện hiệu suất và độ tin cậy của hệ thống robot.

Ta tiến hành vẽ một căn phòng có tường bao quanh với kích thước thử nghiệm như sau: 8m x 3,5m x 0,5m. Độ dày tường là 0,15m. Bên trong phòng có 4 khối vật cản với kích thước: 2m x 0,5m x 0,5m.



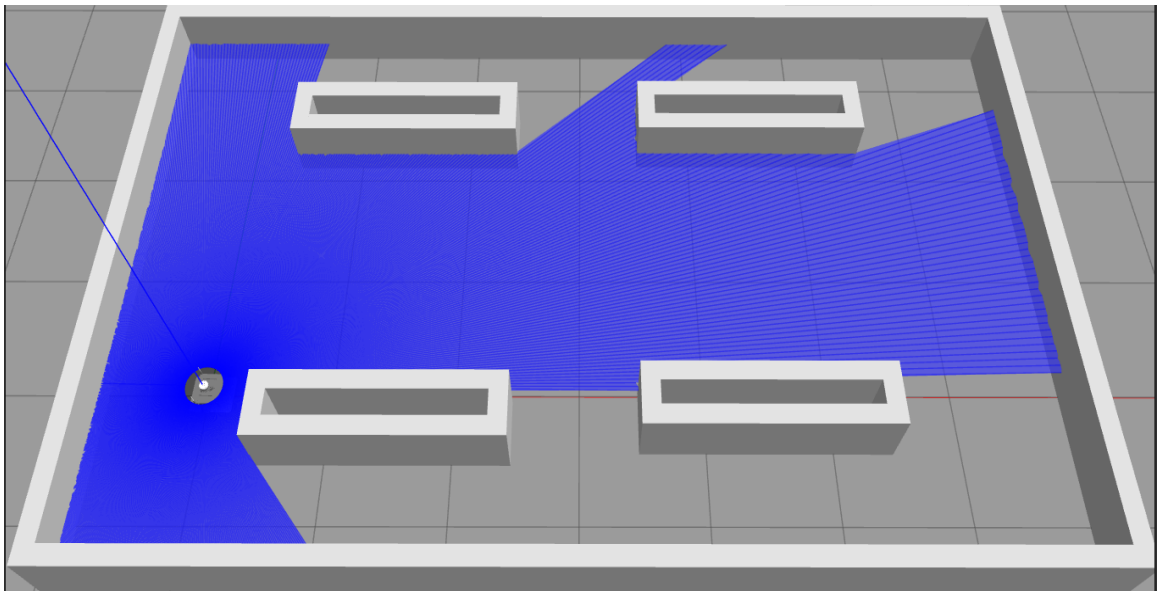
Hình 27: Robot trong không gian hoạt động mô phỏng bằng Gazebo.

4.2.2. Thông số cảm biến lidar

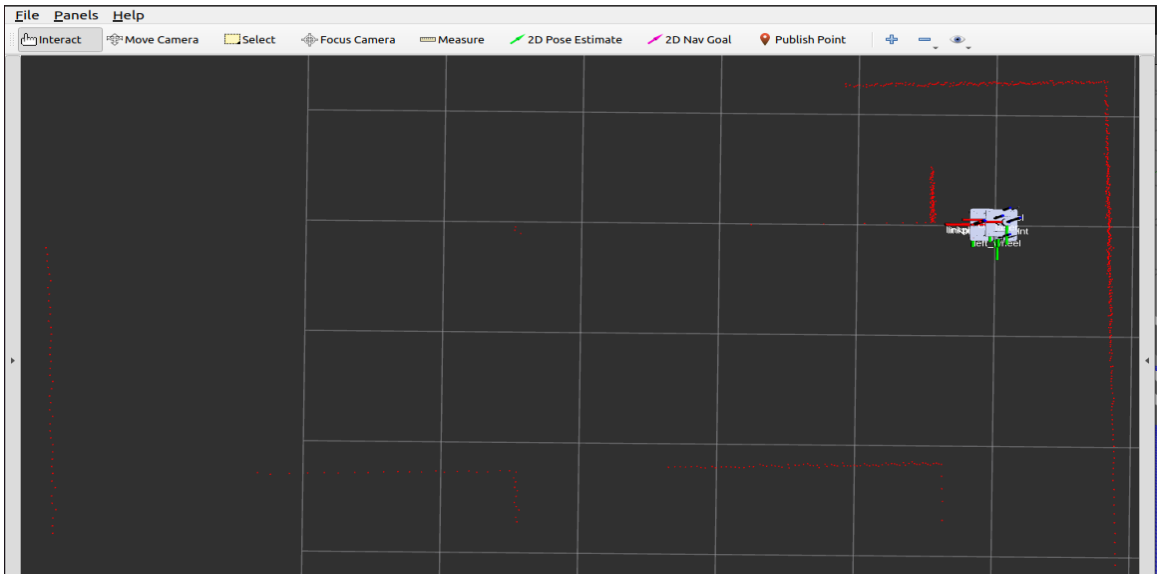
Cảm biến lidar lấy thông số theo RPLidar A1M8 mô phỏng nhiễu gaussian với các giá trị như sau:

Bảng 8. Thông số lựa chọn cho cảm biến lidar mô phỏng.

Thông số	Giá trị
Khoảng cách quét	10m
Góc quét	360°
Độ phân giải	0.1 rad

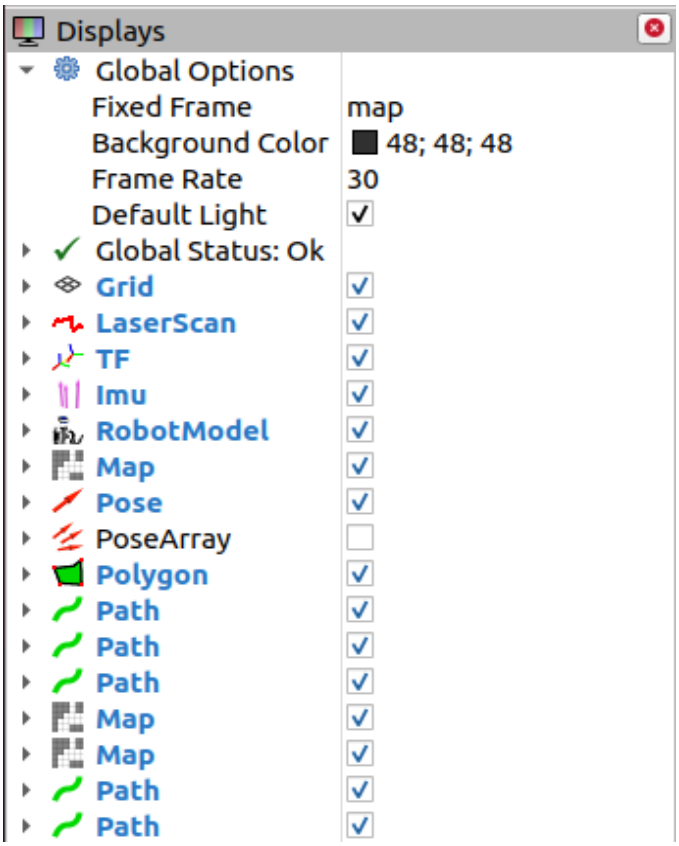


Hình 28: Mô phỏng robot trong môi trường Gazebo.

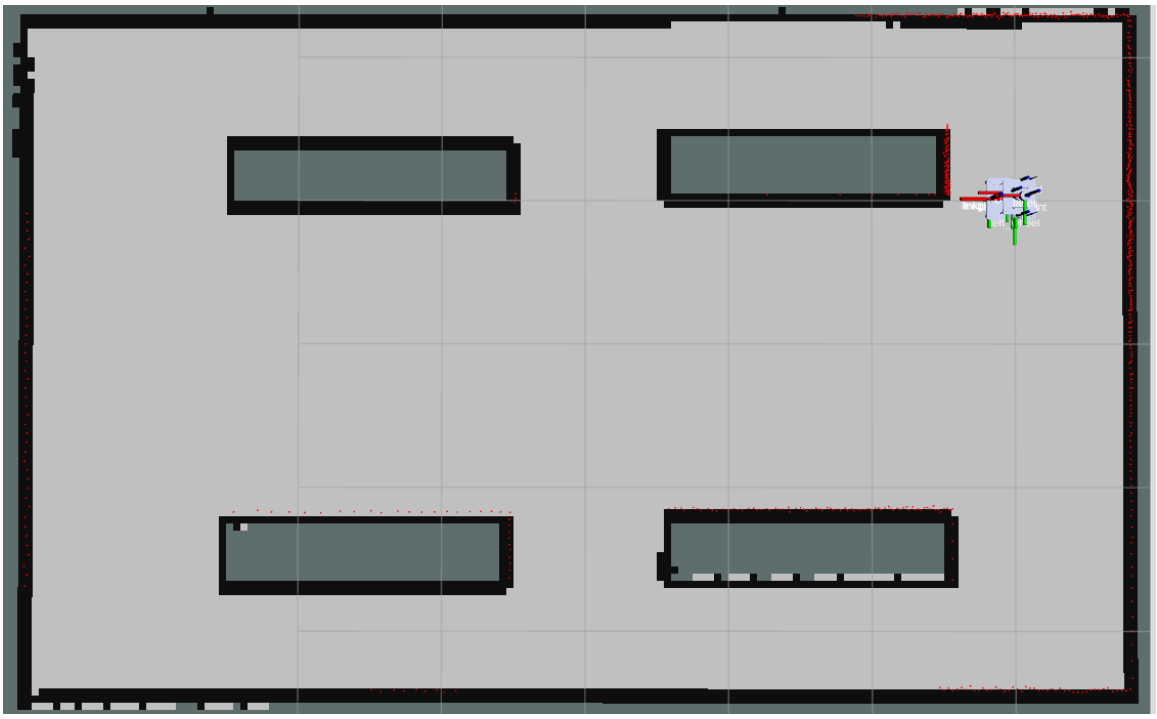


Hình 29: Hiển thị robot và dữ liệu cảm biến lidar trên Rviz.

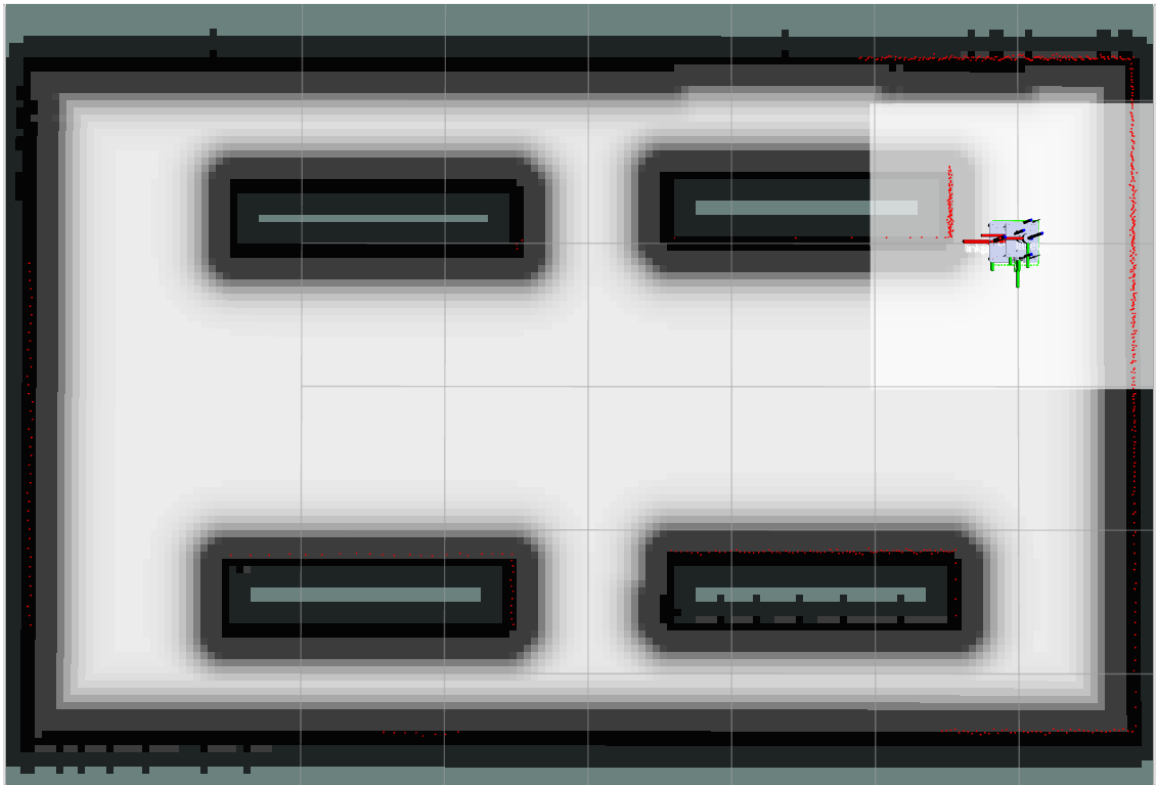
4.3. Quan sát robot thông qua Rviz



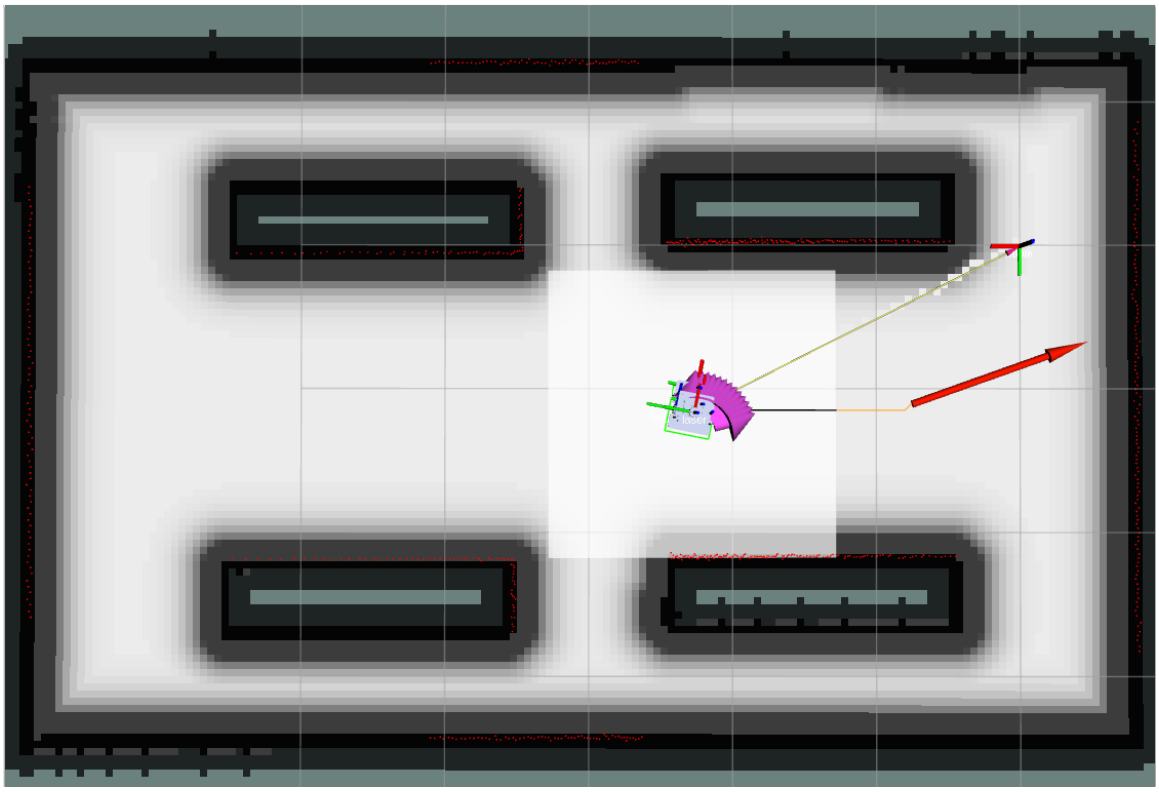
Hình 30: Thêm các chủ đề cần quan sát vào Rviz.



Hình 31: Cửa sổ Rviz hiển thị bản đồ 2D, tín hiệu cảm biến và dữ liệu trạng thái robot khi chưa chạy Move base.

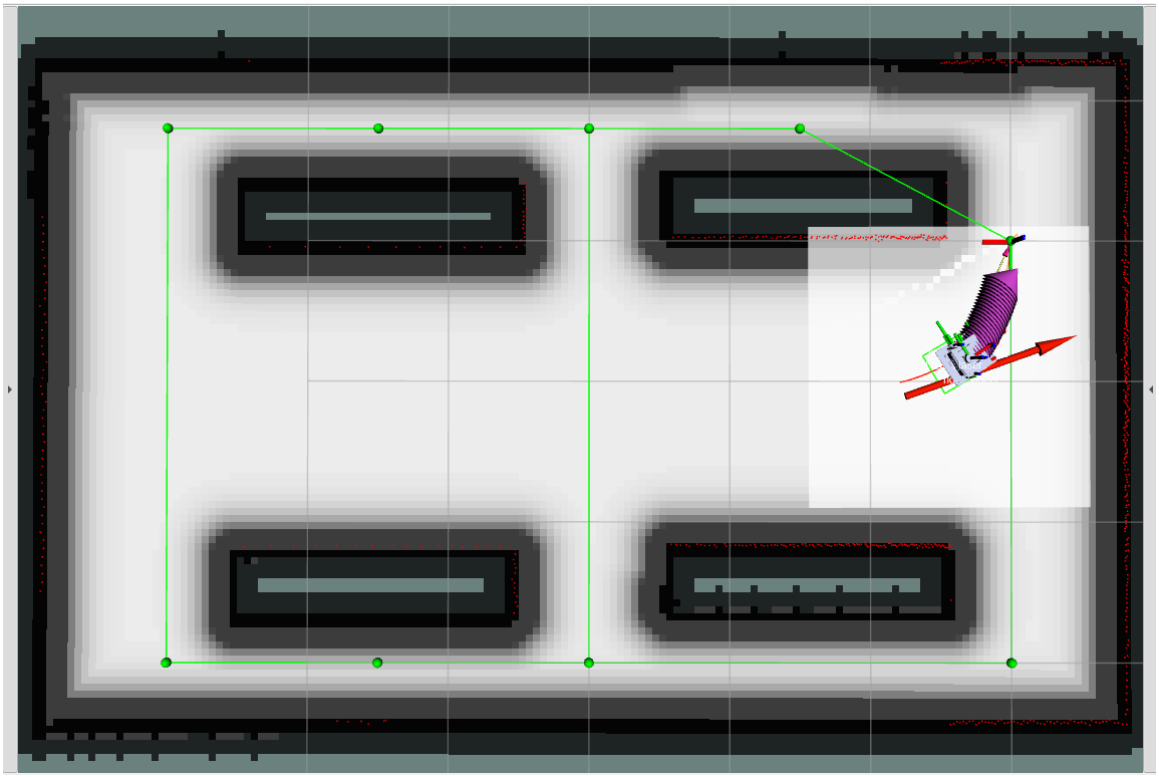


Hình 32: Cửa sổ Rviz hiển thị bản đồ 2D trong lúc chạy Move base.

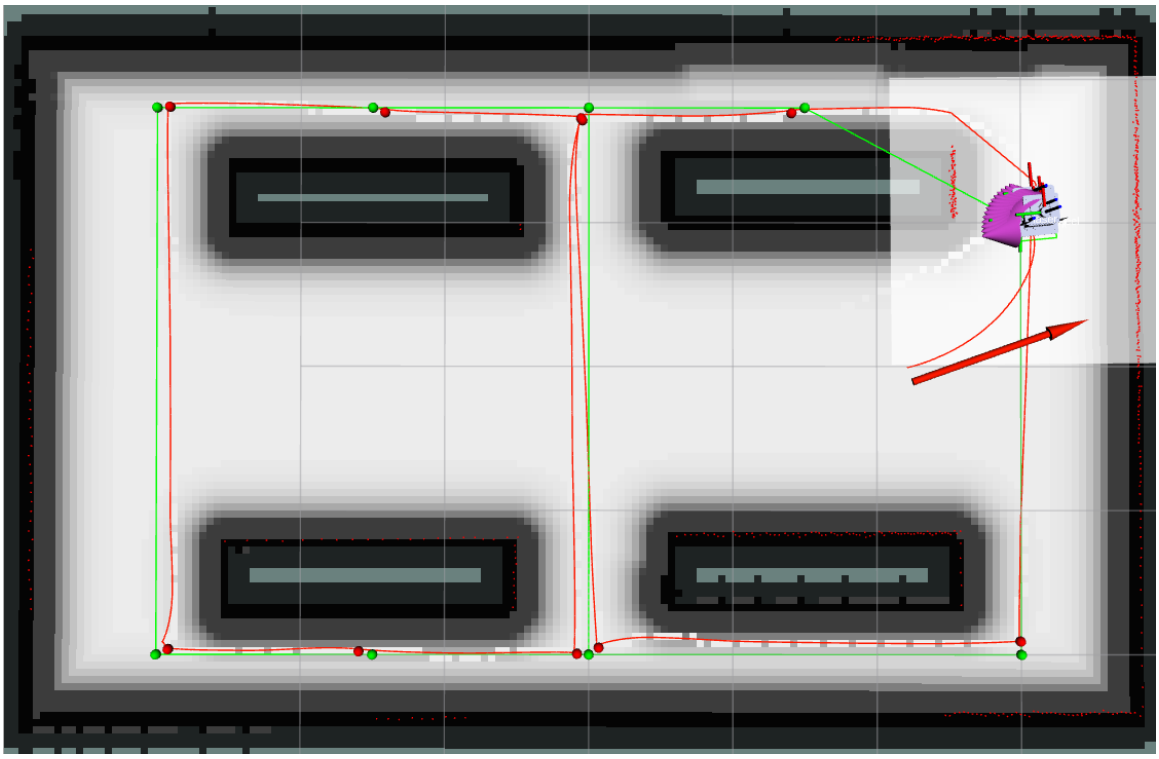


Hình 33: Cửa sổ Rviz hiển thị điểm đơn mục tiêu 2D Nav Goal và quỹ dạo tính toán của robot.

Đường màu đen thể hiện tính toán Local_planner; Đường màu vàng thể hiện tính toán Global_planner.



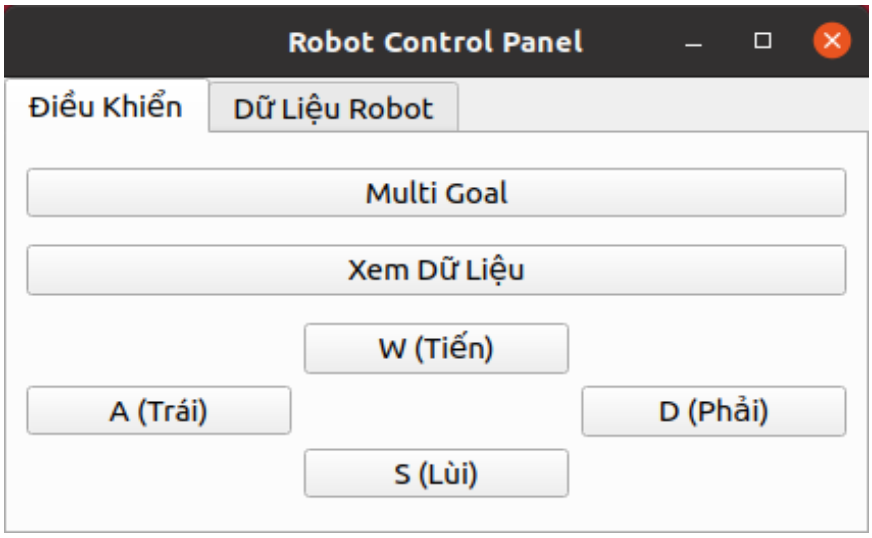
Hình 34: Cửa sổ Rviz hiển thị điểm mục tiêu Multi - Goal và quỹ đạo mong muốn.



Hình 35: Cửa sổ Rviz hiển thị điểm mục tiêu Multi - Goal đã xác nhận và quỹ đạo đã đi được.

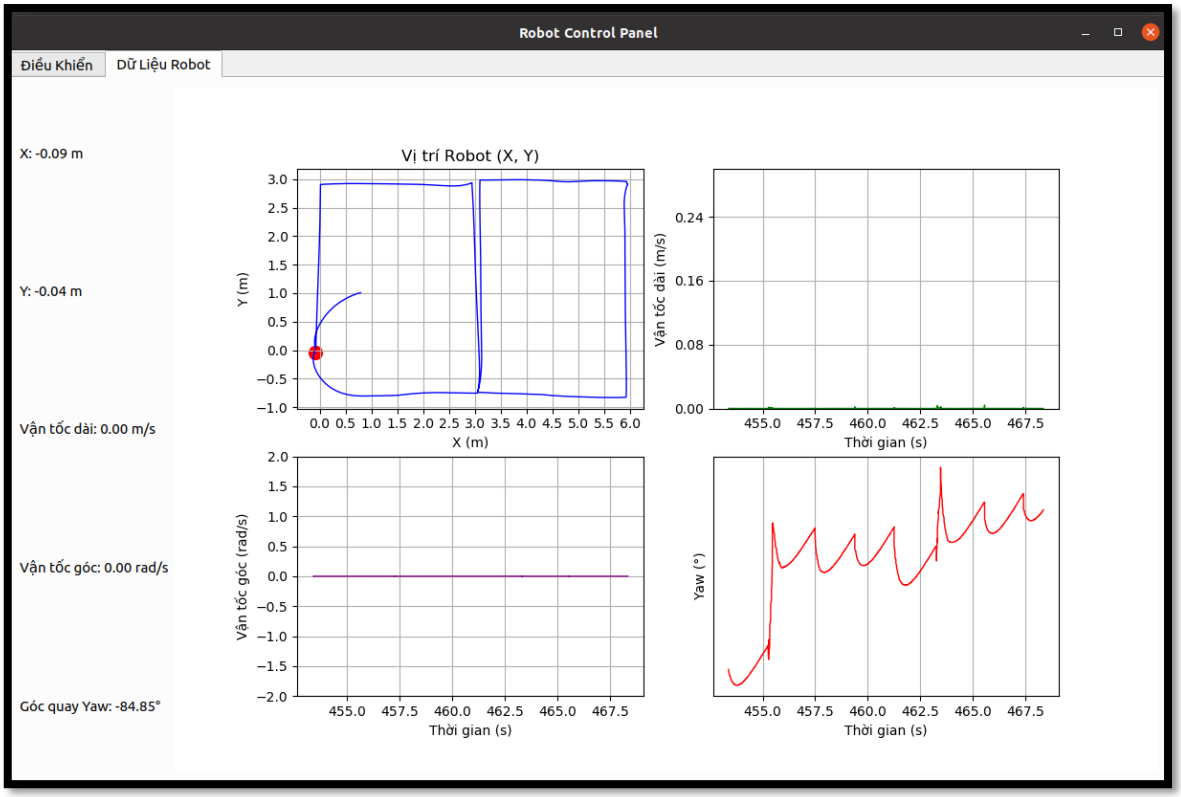
Đường màu xanh và chấm tròn xanh thể hiện quỹ đạo đặt cho robot và điểm mục tiêu; Đường màu đỏ và chấm tròn đỏ thể hiện quỹ đạo đi thực tế và điểm đến đã xác nhận so với điểm mục tiêu.

4.4. Thiết kế giao diện điều khiển và thu thập dữ liệu bằng PyQt5



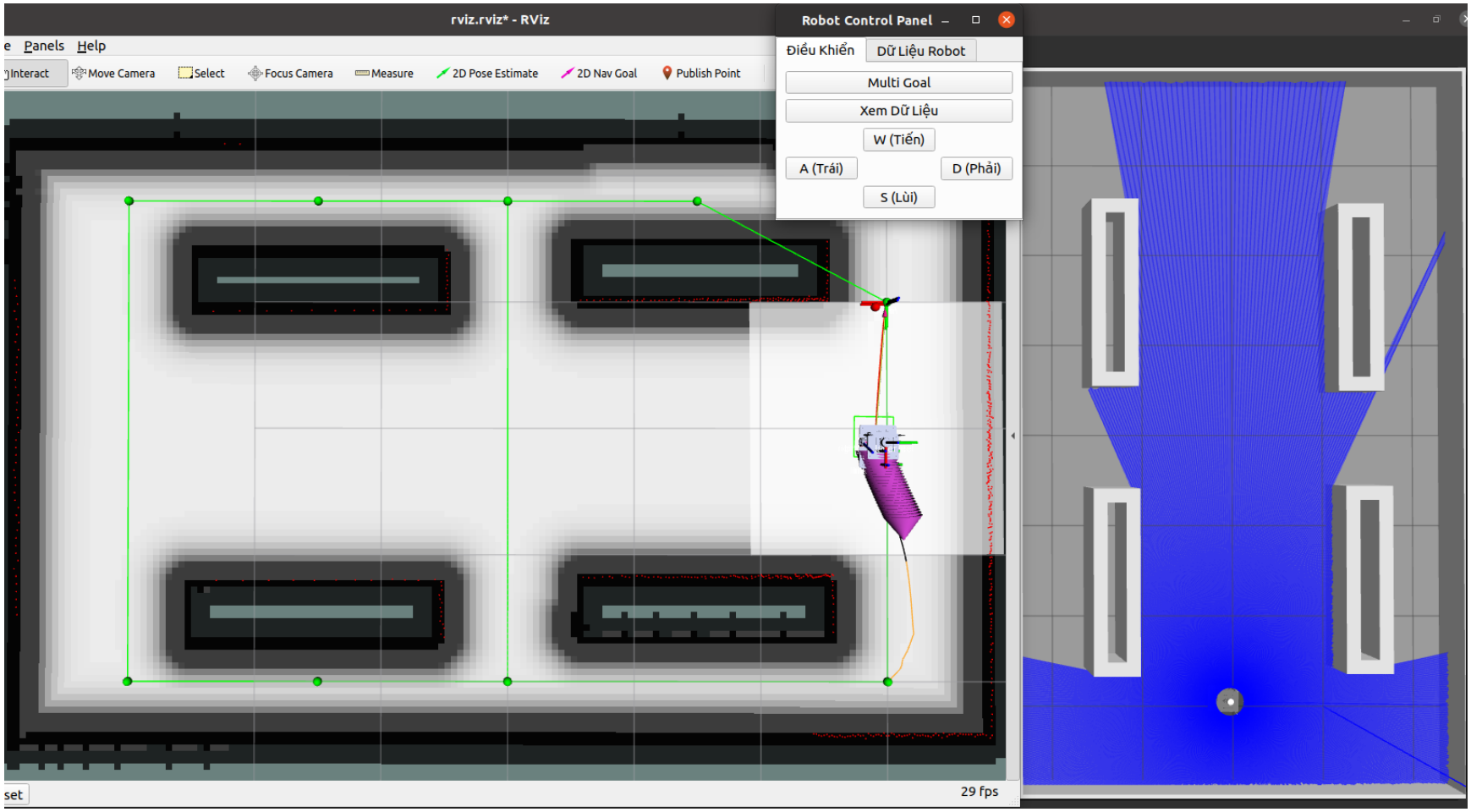
Hình 36: Giao diện người điều khiển và giám sát dữ liệu.

Giao diện có nút "Xem dữ liệu", cho phép người giám sát có thể chuyển qua tab dữ liệu dễ dàng. Khi nhấn nút của sổ dữ liệu sẽ tự động phóng to ra. Để thoát trạng thái phóng to, ta chỉ cần nhấn Esc.



Hình 37: Cửa sổ dữ liệu thu thập từ trạng thái của robot.

Ta tiến hành thu thập dữ liệu từ các topic ở dạng số, sau đó xử lý và hiển thị dưới dạng đồ thị. Với biểu đồ vị trí, ta sử dụng biểu đồ với hệ tọa độ Oxy, khi đó đồ thị đường đi của robot sẽ là quỹ đạo vẽ ra trên thực tế. Chấm đỏ thể hiện vị trí hiện tại của robot trên bản đồ. Đối với các biểu đồ khác, đồ thị sẽ dịch sang trái khi thời gian vượt quá khả năng hiển thị là 15 giây gần nhất.



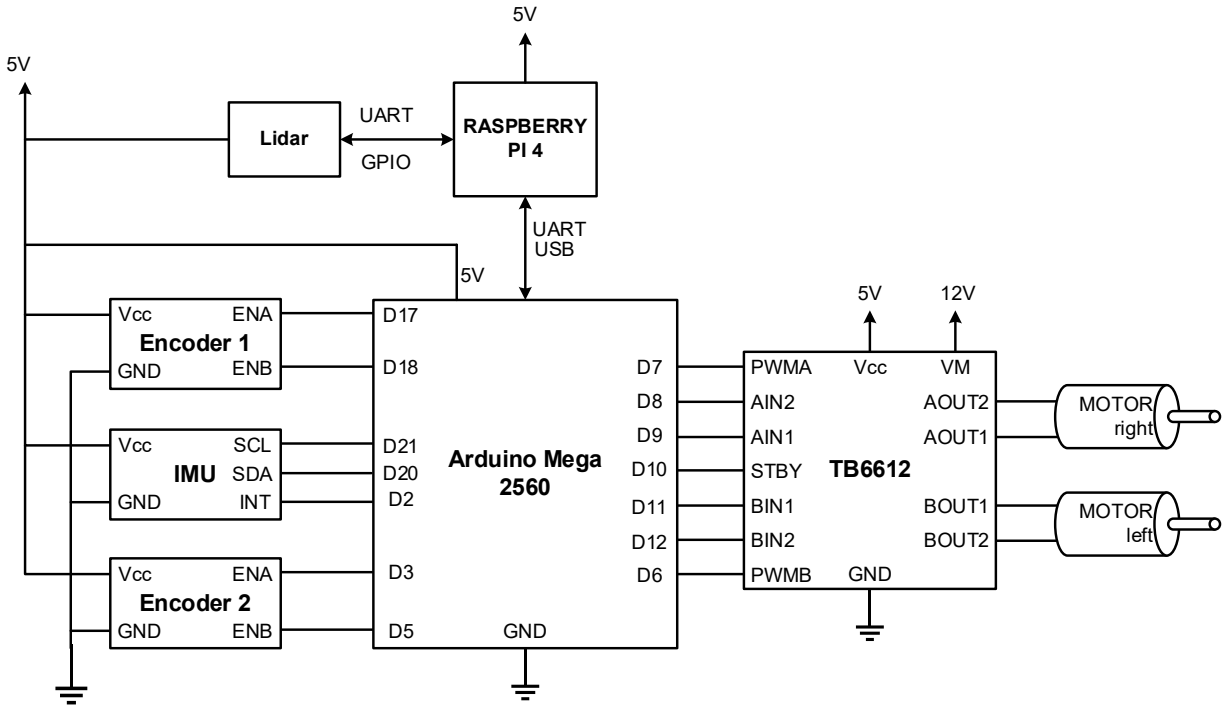
Hình 38: Bố trí các cửa sổ trong quá trình điều khiển robot.

Đường màu xanh và chấm tròn xanh thể hiện quỹ đạo đặt cho robot và điểm mục tiêu; Đường màu đỏ và chấm tròn đỏ thể hiện quỹ đạo đi thực tế và điểm đến đã xác nhận so với điểm mục tiêu.

Chương 5:
THI CÔNG MÔ HÌNH

Sau khi mô phỏng đạt kết quả như ý muốn, ta sẽ sử dụng bộ điều khiển và thông số cài đặt như trên mô phỏng để tiến hành thi công và điều khiển cho robot thật. Lúc này ta xem như cơ cấu chấp hành không còn là robot trên Gazebo nữa, mà là robot thực tế. Bên cạnh đó, dữ liệu mà Gazebo đã đảm nhận phát ra trong quá trình mô phỏng sẽ được chuyển sang cho robot thật tính toán và publish lên hệ thống ROS thông qua topic.

5.1. Sơ đồ mạch điện



Hình 39: Sơ đồ mạch điện robot.

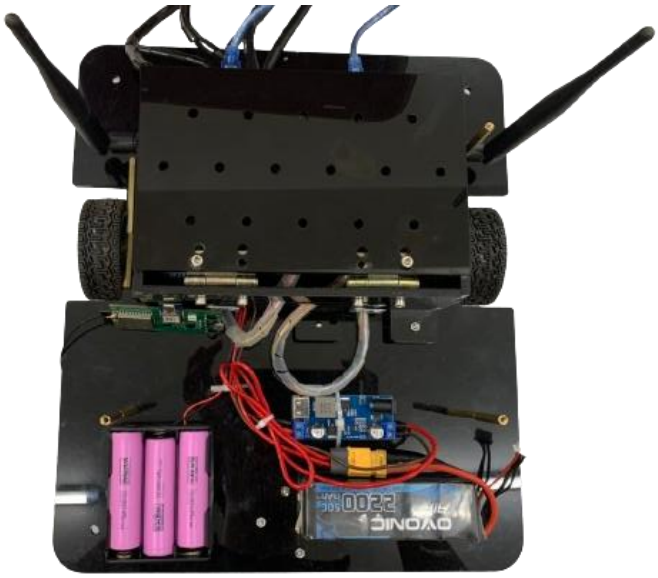
Ở mạch này ta có các lưu ý sau:

- Cảm biến lidar được kết nối với Raspberry theo cách trực tiếp, tức là cắm thẳng lên các chân GPIO. Do đó, nguồn cấp cho cảm biến Lidar hoạt động sẽ là nguồn ngoài, không dùng nguồn trên máy tính nhúng.

- Các chân cấp nguồn và điều khiển cho 2 động cơ được nối đúng theo thiết kế mặc định của motor, do đó cần đổi dấu chiều động cơ bên phải trong phần tính toán.

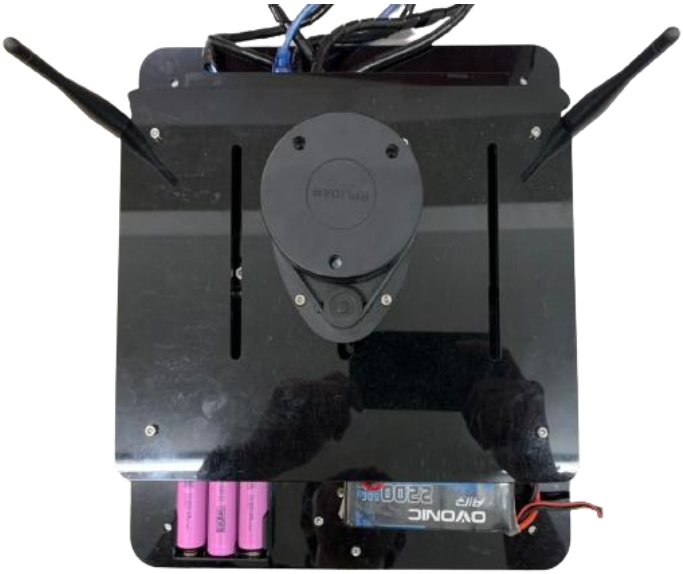
5.2. Lắp ráp phần cứng.

- Lắp ráp tầng một (base_link) cho robot:



Hình 40: Khung tầng một base_link.

- Lắp ráp tầng hai (lidar_link):



Hình 41: Khung tầng hai lidar_link.

5.3. Giao tiếp dữ liệu

5.3.1. Giao tiếp giữa Arduino và Raspberry

Vi điều khiển Arduino và máy tính nhúng Raspberry Pi là hai thiết bị phổ biến và có sẵn các thư viện giúp kết nối dữ liệu thông qua USB.

Cả hai thiết bị đều được gắn trên robot khi di chuyển, đầu tiên ta kết nối bằng cáp USB:

Hình 42: Kết nối vi điều khiển và máy tính nhúng bằng cáp USB.

Cấu hình mạng cho VMware:

```
export ROS_MASTER_URI=http://192.168.41.146:11311
export ROS_IP=192.168.41.132
```

Bật ROS MASTER trên VMware: `roscore`

Tiếp theo ta chạy node giao tiếp trên Raspberry như sau:

```
rosrun rosserial_python serial_node.py _port:=/dev/ttyACM0 _baud:=115200
```

```
nttien@raspi:~$ rosrun rosserial_python serial_node.py _port:=/dev/ttyACM0 _baud:=115200
[INFO] [1750414617.525316]: ROS Serial Python Node
[INFO] [1750414617.580642]: Connecting to /dev/ttyACM0 at 115200 baud
[INFO] [1750414619.693427]: Requesting topics...
[INFO] [1750414619.751726]: Note: publish buffer size is 512 bytes
[INFO] [1750414619.758794]: Setup publisher on encoder_ticks [std_msgs/Int32MultiArray]
[INFO] [1750414619.865063]: Setup publisher on imu/data [sensor_msgs/Imu]
[INFO] [1750414619.891586]: Note: subscribe buffer size is 512 bytes
[INFO] [1750414619.902558]: Setup subscriber on cmd_vel [geometry_msgs/Twist]
```

Hình 43: Thông báo kết nối dữ liệu thành công.

Trong đó ta có thể tùy chỉnh tốc độ baud làm sao cho tín hiệu là ổn định nhưng vẫn nằm trong khoảng cho phép của phần cứng, ở đây nhóm sử dụng Arduino Mega nên có thể đặt tốc độ baud là 115200.

Để gửi dữ liệu Encoder và IMU lên Raspberry, ta dùng chương trình sau:

Chương trình Arduino IDE:

```
#include <ros.h>
#include <geometry_msgs/Twist.h>
#include <std_msgs/Int32MultiArray.h>
```

```
#include <sensor_msgs/Imu.h>
#include <Encoder.h>
#include <Wire.h>
#include <MPU6050.h>
// ==== Motor Pins ====
#define PWMB 6
#define BIN1 11
#define BIN2 12
#define PWMA 7
#define AIN1 9
#define AIN2 8
#define STBY 10
#define INVERT_LEFT_MOTOR true
#define INVERT_RIGHT_MOTOR true
// ==== Encoder ====
Encoder encoderLeft(3, 5);
Encoder encoderRight(18, 17);
const long COUNTS_PER_REV = 11880;
// === MPU6050 ====
MPU6050 imu;
int16_t ax, ay, az;
int16_t gx, gy, gz;
// ==== Robot Constants ====
const float WHEEL_BASE = 0.232;
const float MAX_SPEED = 0.5;
const int MIN_PWM = 50;
// ==== ROS ====
ros::NodeHandle nh;
// Encoder message
std_msgs::Int32MultiArray encoder_msg;
ros::Publisher encoder_pub("encoder_ticks", &encoder_msg);
// IMU message
sensor_msgs::Imu imu_msg;
ros::Publisher imu_pub("imu/data", &imu_msg);
// ==== Motor Control ====
void setMotor(int pwm_pin, int in1, int in2, float speed, bool invert) {
  int pwm = constrain(abs(speed), 0, 255);
  if (pwm < MIN_PWM && pwm > 0) pwm = MIN_PWM;
  bool dir = (speed >= 0);
  if (invert) dir = !dir;
  digitalWrite(in1, dir ? HIGH : LOW);
  digitalWrite(in2, dir ? LOW : HIGH);
  analogWrite(pwm_pin, pwm);
}
void cmdVelCb(const geometry_msgs::Twist& msg) {
  float v = msg.linear.x;
  float omega = msg.angular.z;
  float v_left = v - (omega * WHEEL_BASE / 2.0);
```

```
  float v_right = v + (omega * WHEEL_BASE / 2.0);
  float pwm_l = v_left * 255.0 / MAX_SPEED;
  float pwm_r = v_right * 255.0 / MAX_SPEED;
  setMotor(PWMB, BIN1, BIN2, pwm_l, INVERT_LEFT_MOTOR);
  setMotor(PWMA, AIN1, AIN2, pwm_r, INVERT_RIGHT_MOTOR);
}
ros::Subscriber<geometry_msgs::Twist> sub("cmd_vel", cmdVelCb);
// ==== Setup ====
void setup() {
  pinMode(PWMB, OUTPUT); pinMode(BIN1, OUTPUT); pinMode(BIN2, OUTPUT);
  pinMode(PWMA, OUTPUT); pinMode(AIN1, OUTPUT); pinMode(AIN2, OUTPUT);
  pinMode(STBY, OUTPUT); digitalWrite(STBY, HIGH);
  Serial.begin(115200);
  nh.getHardware()->setBaud(115200);
  Wire.begin();
  imu.initialize();
  nh.initNode();
  nh.subscribe(sub);
  encoder_msg.data_length = 3;
  encoder_msg.data = new int32_t[3];
  nh.advertise(encoder_pub);
  nh.advertise(imu_pub);
}
// ==== Loop ====
void loop() {
  // === Encoder ===
  long left = encoderLeft.read();
  long right = encoderRight.read();
  long now = millis();
  encoder_msg.data[0] = left;
  encoder_msg.data[1] = right;
  encoder_msg.data[2] = now;
  encoder_pub.publish(&encoder_msg);
  // === IMU ===
  imu.getMotion6(&ax, &ay, &az, &gx, &gy, &gz);
  imu_msg.linear_acceleration.x = ax / 16384.0;
  imu_msg.linear_acceleration.y = ay / 16384.0;
  imu_msg.linear_acceleration.z = az / 16384.0;
  imu_msg.angular_velocity.x = gx / 131.0;
  imu_msg.angular_velocity.y = gy / 131.0;
  imu_msg.angular_velocity.z = gz / 131.0;
  imu_msg.orientation_covariance[0] = -1;
  imu_msg.header.stamp = nh.now();
  imu_msg.header.frame_id = "IMU";  // chuẩn frame gốc cho IMU
  imu_pub.publish(&imu_msg);
  nh.spinOnce();
  delay(0.1);  //10= ~100 Hz
}
```

5.3.2. Giao tiếp giữa Raspberry và VMware

Đây là 2 thiết bị trong hệ thống ROS, ta sẽ cấu hình cho VMware Ubuntu là máy cố định và có cấu hình mạnh làm ROS MASTER đảm nhiệm các chức năng tính toán, hiển thị và quản lý dữ liệu. Trong khi đó, Raspberry làm ROS SLAVE vì cần di động và trung gian cho Arduino đọc lệnh từ */cmd_vel*, đồng thời thực hiện công việc nhẹ nhàng hơn là chạy node tổng hợp dữ liệu gửi lên roscore thông qua topic.

Đầu tiên, ta kết nối cả 2 thiết bị vào chung một mạng wifi, đây là bước quan trọng. Robot và máy tính giao tiếp từ xa thông qua wifi, cũng tức là Raspberry và VMware giao tiếp thông qua wifi.

Sau khi kết nối chung mạng, ta tiến hành kiểm tra IP kết nối mạng của Raspberry và VMware, sau đó SSH giữa hai máy với VMware là server chủ.

Trên máy tính, mở **Command Prompt**, chạy lệnh:

```
ssh nttien@192.168.41.132
```

```
C:\Users\User>ssh nttien@192.168.41.132
nttien@192.168.41.132's password:
Linux raspi 6.1.21-v8+ #1642 SMP PREEMPT Mon Apr  3 17:24:16 BST 2023 aarch64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Jun 20 17:42:51 2025 from 192.168.41.199
```

Hình 44: SSH thành công từ máy tính đến Raspberry.

Cấu hình mạng cho Raspberry:

```
export ROS_MASTER_URI=http://192.168.41.146:11311
export ROS_IP=192.168.41.132
```

Lúc này, chạy node python được viết để tính động học cho robot, các dữ liệu về trạng thái của robot sẽ được đăng tải lên roscore từ Raspberry thay cho Gazebo trước đây. Mở một terminal khác và chạy chương trình động học sau:

Chương trình encoder_to_odom.py:

```
#!/usr/bin/env python3
import rospy, math
from std_msgs.msg import Int32MultiArray
from nav_msgs.msg import Odometry
from geometry_msgs.msg import Quaternion, TransformStamped
import tf2_ros
class EncoderOdomNode:
```

```
def __init__(self):
    rospy.init_node('encoder_to_odom')
    # decoder tick topic
    rospy.Subscriber('encoder_ticks', Int32MultiArray, self.tick_cb)
    self.odom_pub = rospy.Publisher('odom', Odometry, queue_size=10)
    # TF static broadcaster for base_link→imu_link (example)
    self.tf_broadcaster = tf2_ros.TransformBroadcaster()
    # robot parameters
    self.wheel_base = rospy.get_param('~wheel_base', 0.232)
    self.ticks_per_rev = rospy.get_param('~ticks_per_rev', 11880)
    self.wheel_radius = rospy.get_param('~wheel_radius', 0.034)
    # state
    self.prev_left = None
    self.prev_right = None
    self.prev_t = None
    self.x = self.y = self.theta = 0.0
def tick_cb(self, msg):
    left, right, t_ms = msg.data
    t = t_ms / 1000.0
    if self.prev_t is None:
        self.prev_left = left
        self.prev_right = right
        self.prev_t = t
        return
    dt = t - self.prev_t
    if dt <= 0:
        return
    # compute distances
    d_left = left - self.prev_left
    d_right = right - self.prev_right
    dist_l = 2 * math.pi * self.wheel_radius * d_left / self.ticks_per_rev
    dist_r = 2 * math.pi * self.wheel_radius * d_right / self.ticks_per_rev
    dist = (dist_r + dist_l) / 2
    dtheta = (dist_r - dist_l) / self.wheel_base
    # update pose
    self.theta += dtheta
    self.x += dist * math.cos(self.theta)
    self.y += dist * math.sin(self.theta)
    vx = dist / dt
    vy = 0.0  # planar motion, vy=0; you may opt to calculate lateral ref frame
    v = math.sqrt(vx**2 + vy**2)
    w = dtheta / dt
    # publish odom
    od = Odometry()
    od.header.stamp = rospy.Time.from_sec(t)
    od.header.frame_id = 'odom'
    od.child_frame_id = 'base_link'
    od.pose.pose.position.x = self.x
    od.pose.pose.position.y = self.y
    od.pose.pose.position.z = 0.0
```

```
        q = Quaternion()
        q.z = math.sin(self.theta / 2)
        q.w = math.cos(self.theta / 2)
        od.pose.pose.orientation = q
        od.twist.twist.linear.x = vx
        od.twist.twist.linear.y = vy
        od.twist.twist.angular.z = w
        self.odom_pub.publish(od)
        # broadcast static imu_link if needed
        T = TransformStamped()
        T.header.stamp = od.header.stamp
        T.header.frame_id = 'base_link'
        T.child_frame_id = 'IMU'
        T.transform.translation.x = 0
        T.transform.translation.y = 0
        T.transform.translation.z = 0
        T.transform.rotation = Quaternion(0, 0, 0, 1)
        self.tf_broadcaster.sendTransform(T)
        # update prev
        self.prev_left, self.prev_right, self.prev_t = left, right, t
if _name_ == '_main_':
    node = EncoderOdomNode()
    rospy.spin()
```

```
nttien@raspi:~$ rosrun encoder_odom_pkg encoder_to_odom.py
dl=7 dr=9 dist_l=0.000 dist_r=0.000 dist=0.000 dtheta=0.000 vx=0.000
dl=18 dr=26 dist_l=0.000 dist_r=0.000 dist=0.000 dtheta=0.001 vx=0.013
dl=28 dr=39 dist_l=0.001 dist_r=0.001 dist=0.001 dtheta=0.001 vx=0.020
dl=35 dr=46 dist_l=0.001 dist_r=0.001 dist=0.001 dtheta=0.001 vx=0.024
dl=43 dr=56 dist_l=0.001 dist_r=0.001 dist=0.001 dtheta=0.001 vx=0.029
dl=47 dr=58 dist_l=0.001 dist_r=0.001 dist=0.001 dtheta=0.001 vx=0.030
dl=51 dr=60 dist_l=0.001 dist_r=0.001 dist=0.001 dtheta=0.001 vx=0.034
dl=55 dr=63 dist_l=0.001 dist_r=0.001 dist=0.001 dtheta=0.001 vx=0.035
dl=58 dr=65 dist_l=0.001 dist_r=0.001 dist=0.001 dtheta=0.001 vx=0.037
dl=61 dr=67 dist_l=0.001 dist_r=0.001 dist=0.001 dtheta=0.000 vx=0.038
dl=63 dr=68 dist_l=0.001 dist_r=0.001 dist=0.001 dtheta=0.000 vx=0.039
dl=64 dr=69 dist_l=0.001 dist_r=0.001 dist=0.001 dtheta=0.000 vx=0.040
dl=65 dr=70 dist_l=0.001 dist_r=0.001 dist=0.001 dtheta=0.000 vx=0.040
dl=67 dr=70 dist_l=0.001 dist_r=0.001 dist=0.001 dtheta=0.000 vx=0.042
```

Hình 45: Kết quả chạy thành công node tính odom.

Lúc này, ta thấy in ra màn hình là dữ liệu thô. Đây là dữ liệu gửi thẳng vào thay thế cho các dữ liệu mà trước đó Gazebo phát ra. Do đó, ta có thể dùng lại toàn bộ những giao diện và đồ thị theo dõi robot đã được làm trong phần mô phỏng trước đây, gồm có Rviz và PyQt5.

Đến đây, hệ thống robot hai bánh vi sai điều khiển qua ROS đã hoàn chỉnh kết nối dữ liệu và giao tiếp. Có thể tiến hành điều khiển trên VMware.

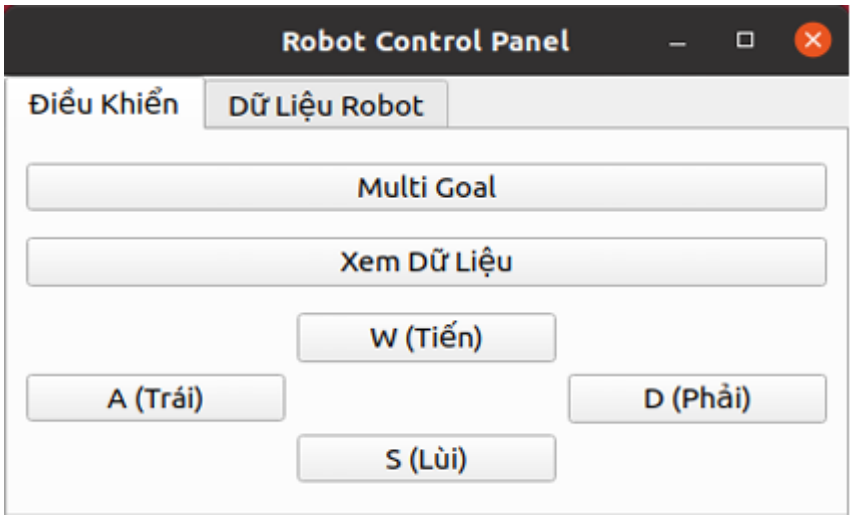
5.3.3. Kết nối Raspberry với cảm biến Lidar

(Nhóm chưa thực hiện được)

Chương 6:
THỰC NGHIỆM VÀ KẾT LUẬN

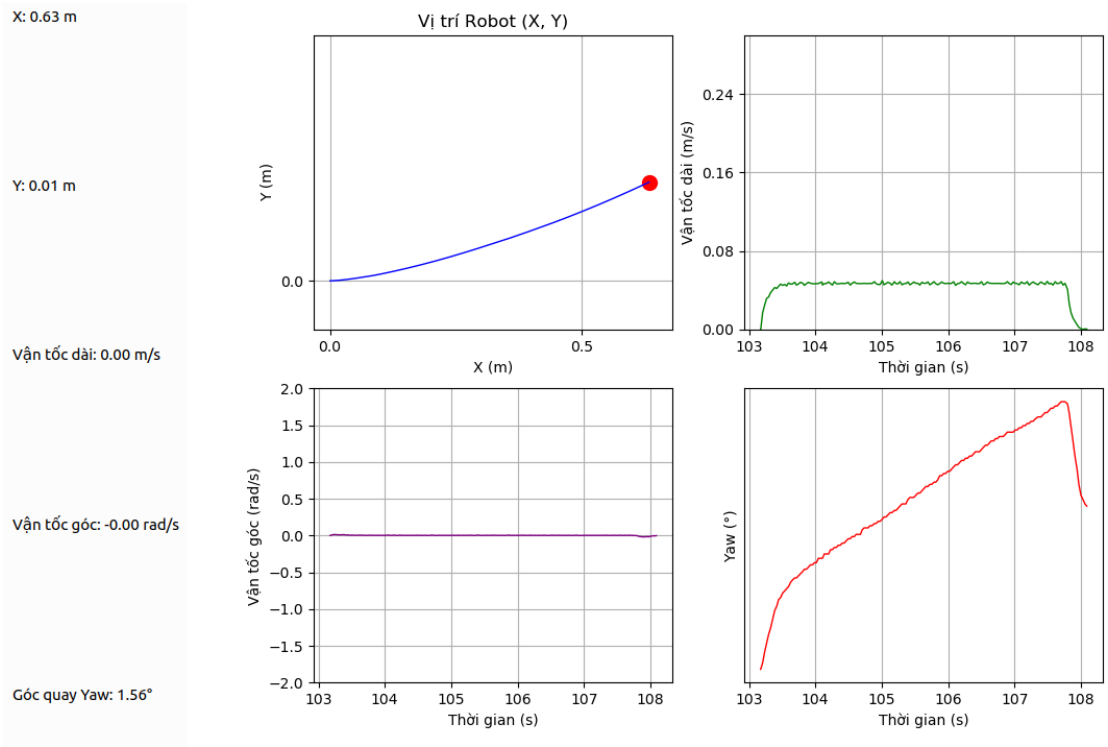
6.1. Thực nghiệm điều khiển bằng lệnh đơn

Robot đã có thể nhận lệnh từ topic /cmd_vel, do đó bất kỳ câu lệnh hay chương trình nào từ VMware phát tín hiệu thì robot đều có thể di chuyển. Ở đây, nhóm dùng lại giao diện người dùng đã xây dựng ở phần mô phỏng trước đó.



Hình 46: Giao diện điều khiển robot.

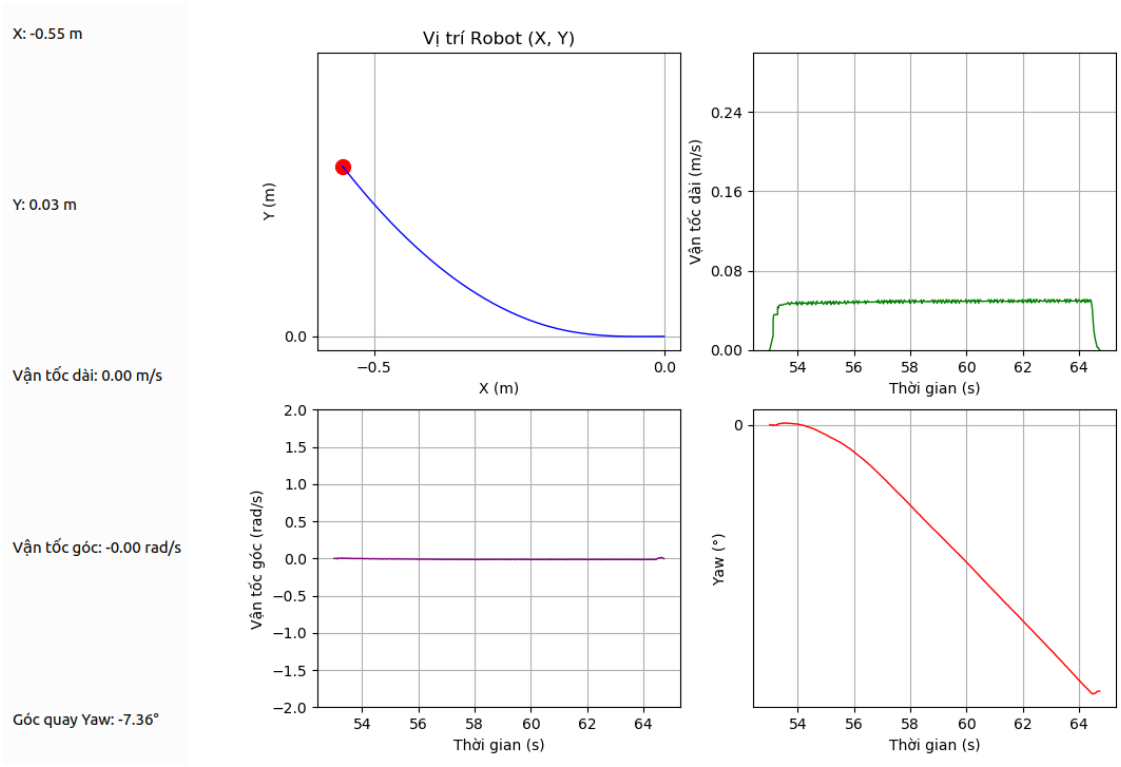
*** Thử nghiệm 1:** Nhận lệnh W, đi tới.



Hình 47: Dữ liệu điều khiển robot đi tới.

- Nhận xét: Ta quan sát thấy khi di chuyển, hai bánh của robot quay không đều dù nhận cùng một lệnh tốc độ. Hiển thị trên đồ thị tọa độ là giá trị y tăng lên 0.01m, sai lệch này tương đối nhỏ so với quảng đường đã đi được trên x là 0.63m nên có thể bỏ qua.

*** Thử nghiệm 2:** Nhận lệnh S, đi lùi.

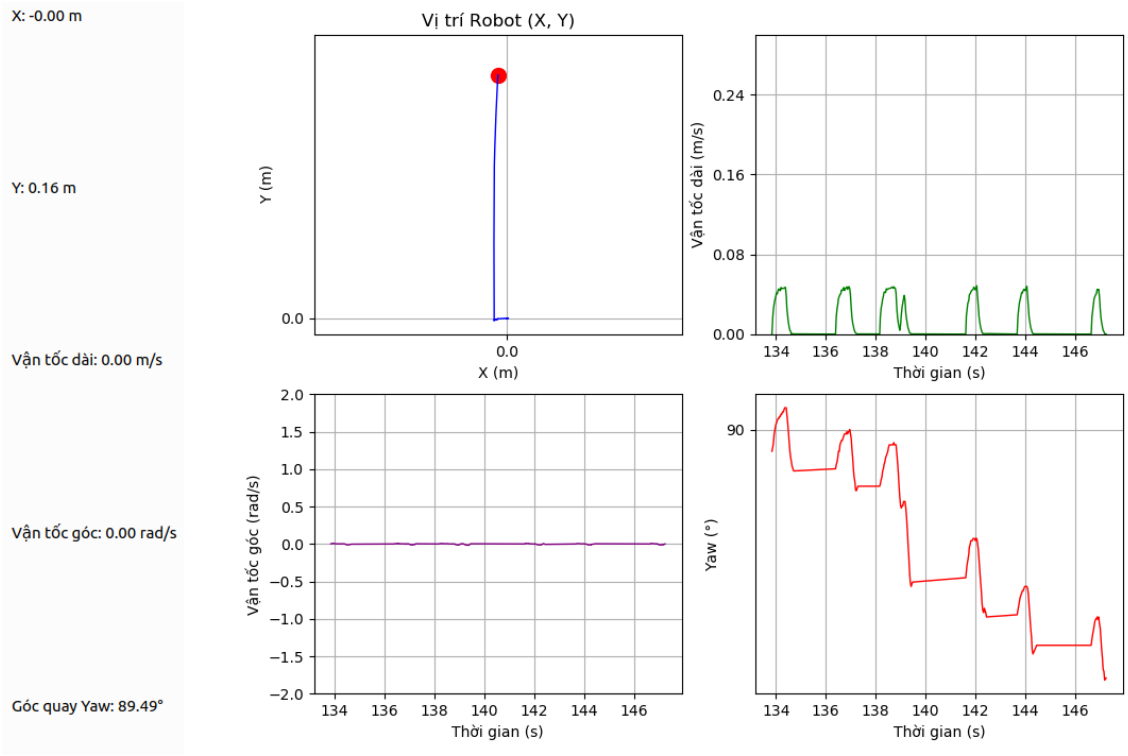


Hình 48: Dữ liệu điều khiển robot đi lùi.

- Nhận xét: Ta quan sát được kết quả tương tự ở thí nghiệm 1. Nhóm đã tiến hành kiểm tra và đúng như nhận xét, bánh phải robot xuất ra nhiều xung hơn bánh trái khoảng 200 xung trên tổng số 11800 xung/vòng. Tức là cùng 1 lệnh điều khiển, bánh phải đi được quãng đường lớn hơn bánh trái, làm cho robot lệch khỏi quỹ đạo thẳng mong muốn một lượng nhỏ không đáng kể.

*** Thử nghiệm 3:** Nhận lệnh A, quay trái.

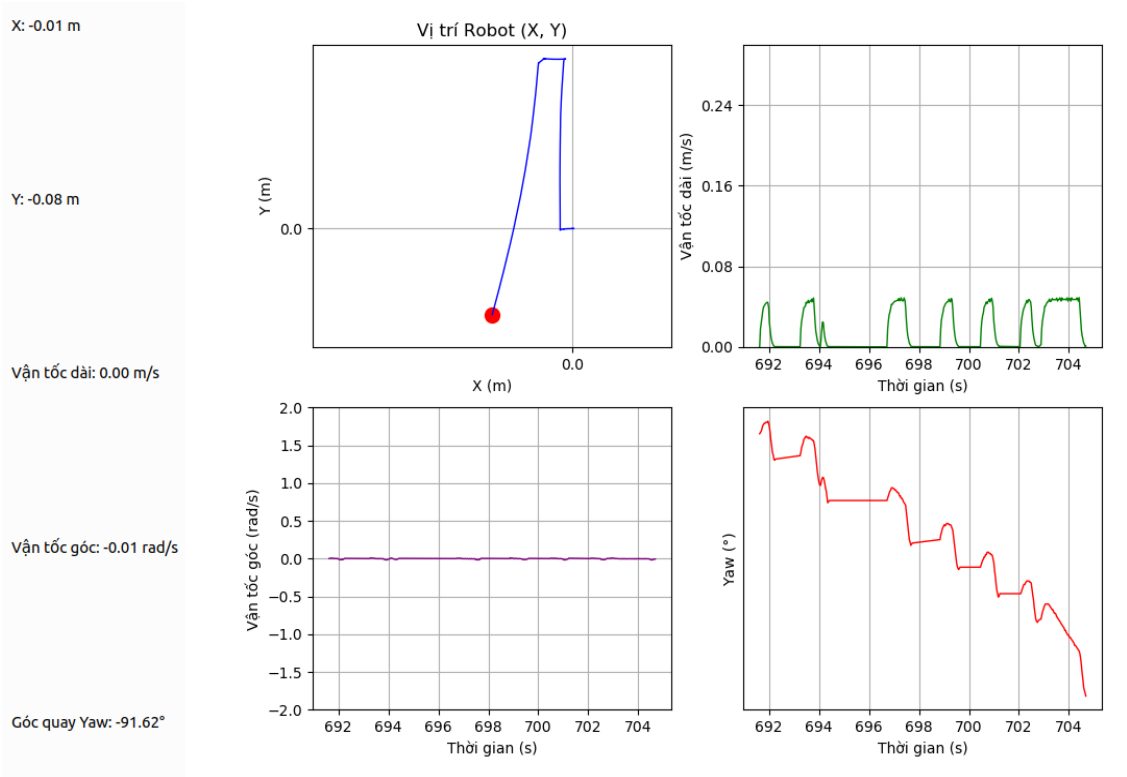
- Để đánh giá được động học của robot có đang đúng theo lý thuyết hay không, nhóm sẽ tiến hành kiểm tra góc quay với giá trị đặt biêt là 90°



Hình 49: Dữ liệu điều khiển robot quay trái (chiều dương góc Yaw của robot).

- Nhận xét: Ta quan sát thấy kết quả ở cả đồ thị chuyển động XY và đồ thị góc Yaw đều cho ra giá trị gần bằng nhau là 90°.

*** Thử nghiệm 4:** Nhận lệnh B, quay phải.



Hình 50: Dữ liệu điều khiển robot quay phải (chiều âm góc Yaw của robot).

- Sử dụng vị trí trước đó của robot ở thử nghiệm 3, nhóm tiến hành điểu khiển để góc quay của robot là -90°.

- Nhận xét: Nhóm kết luận được robot thực tế đã được thiết kế chính xác theo động học robot hai bánh vi sai. Chỉ có một sai số nhỏ ở bánh phải làm cho quãng đường đi được của hai bánh không bằng nhau dẫn đến sai lệch nhỏ ở động học.

6.2. Thực nghiệm điều khiển bằng 2D Nav Goal

(Nhóm chưa thực hiện được)

6.3. Thực nghiệm điều khiển chạy nhiều điểm mục tiêu

(Nhóm chưa thực hiện được)

6.4. Kết luận

Nhóm đã thực hiện thành công mục tiêu nghiên cứu gồm:

- Thiết kế và thi công robot di động hai bánh vi sai.
- Xây dựng môi trường mô phỏng robot.
- Điều khiển robot di động dựa vào ROS.

Qua quá trình thực hiện, nhóm đã thiết kế thành công mô hình mobile robot hai bánh vi sai bằng phần mềm SolidWorks, đảm bảo đầy đủ các chi tiết cơ khí theo yêu cầu. Việc xuất bản vẽ 2D từ mô hình 3D được thực hiện chính xác, đóng vai trò quan trọng làm cơ sở cho giai đoạn mô phỏng sau này.Bên cạnh đó, nhóm cũng đã chuyển đổi thành công mô hình từ SolidWorks sang file URDF tạo điều kiện thuận lợi cho việc mô phỏng điều khiển robot trong Gazebo và hiển thị trực quan trên Rviz.